PORT-AWARE EDGE-FEATURE ENHANCED GRAPH NEURAL NETWORK (PAE-GNN) FOR MULTI-CLASS NETWORK TRAFFIC ANOMALY DETECTION

BY

OYEYEMI, ANUOLUWAPO OLUWATOBILOBA

(22CG031946)

A PROJECT SUBMITTED TO THE DEPARTMENT OF COMPUTER AND INFORMATION SCIENCES, COLLEGE OF SCIENCE AND TECHNOLOGY, COVENANT UNIVERSITY OTA, OGUN STATE.

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE BACHELOR OF SCIENCE (HONOURS) DEGREE IN COMPUTER SCIENCE.

JUNE, 2026

PORT-AWARE EDGE-FEATURE ENHANCED GRAPH NEURAL NETWORK (PAE-GNN) FOR MULTI-CLASS NETWORK TRAFFIC ANOMALY DETECTION

BY

OYEYEMI, ANUOLUWAPO OLUWATOBILOBA

(22CG031946)

DEPARTMENT OF COMPUTER AND INFORMATION SCIENCES, COLLEGE OF SCIENCE AND TECHNOLOGY, COVENANT UNIVERSITY OTA, OGUN STATE.

PROF. OYELADE O. JELILI

SUPERVISOR

CERTIFICATION

I hereby declare that this project was carried out by Oyeyemi, Oluwatobiloba Anuoluwapo in the Department of Computer and Information Sciences, College of Science and Technology, Covenant University, Ota, Ogun State, Nigeria under my supervision.

Prof. Jelili O. Oyelade _____

Supervisor Signature and Date

Prof. Aderonke A. Oni _____

Head of Department Signature and Date

DEDICATION

I dedicate this project to Almighty God for divine guidance, wisdom, and strength through the course of this project. I dedicate this work to my adorable parents, Mr. & Mrs. Oyeyemi for their support, encouragement and sacrifices which is what keeps me going.

ACKNOWLEDGEMENT

Most importantly, I want to thank Almighty God, who has helped me at every stage of the programme and who gave me the wisdom and perseverance necessary to carry out this research project. I am also thankful to my supervisor, Prof. Jelili Oyelade, for his guidance, intellectual input, and constructive criticism during my research.

I would like to thank my lecturers and staff of the Department of Computer and Information Sciences of Covenant University for providing the academic environment for which this study became possible. To my family and friends who believed in me at times when I lost faith, thank you.

TABLE OF CONTENT

CONTENT	PAGES
CERTIFICATION	**iii**
DEDICATION	**iv**
ACKNOWLEDGEMENT	**v**
TABLE OF CONTENT	**vi**
LIST OF FIGURES	**x**
LIST OF TABLES	**xi**
ABBREVIATIONS	**xii**
ABSTRACT	**xiv**
CHAPTER ONE INTRODUCTION	**1**
1.1. Background Information	1
1.2. Statement of the Problem	2
1.3. Aims and Objectives of the Study	3
1.4. Methodology	4
1.5. Significance of the Study	5
1.6. Project Outline	6
CHAPTER TWO LITERATURE REVIEW	**8**
2.1. Preamble	8
2.2. Review of Relevant Concepts	9
2.2.1. Network Security and Intrusion Detection Systems	9
2.2.2. Machine Learning in Cybersecurity	10
2.2.3. Graph Theory and Graph Neural Networks	12
2.3. Theoretical and Conceptual Framework	13
2.4. Review of Related Works	14
2.4.1. E-GraphSAGE: A Graph Neural Network Based Intrusion Detection System	15
2.4.2. Edge-Directed Graph Multi-Head Attention Networks for Network Intrusion Detection	15
2.4.3. A Survey of GNNs for Network Intrusion Detection	16
2.4.4. Anomal-E: Self-Supervised Edge-Based GNN for Anomaly Detection	17
2.4.5. Heterogeneous Graph Neural Network for Intrusion Detection	18
2.4.6. IGNNITION: GNN-Based Traffic Classification with NetFlow Features	18
2.4.7. DÏoT: Federated GNN Anomaly Detection for IoT	19
2.4.8. GCN with Attention for Botnet Detection	20
2.4.9. Simplifying Graph Convolutional Networks for Intrusion Detection	20
2.4.10. Toward Generating a New Intrusion Detection Dataset	21

2.4.11. A Comprehensive Study on GNN Architectures and Applications (Zhou et al., 2020) 22
2.5. Review of Existing Systems 25
2.5.1. Signature-Based NIDS: Snort and Suricata 25
2.5.2. Behavioural Analysis Platform: Zeek 26
2.5.3. Commercial SIEM Platforms 26
2.6. Gaps Identified in Literature 28
2.7. Summary of Literature Review 30
CHAPTER THREE RESEARCH METHODOLOGY 31
3.1. Introduction 31
3.2. Research Design and System Requirements 32
3.2.1. Functional Requirements 33
3.2.2. Non-Functional Requirements 35
3.2.3. System Requirements 37
3.3. Data Collection and Pre-Processing 38
3.3.1. Dataset Acquisition 39
3.3.2. Data Cleaning and Label Consolidation 40
3.3.3. Edge Feature Engineering 40
3.3.4. Class Balancing with SMOTE 43
3.3.5. Graph Construction and Train/Validation/Test Split 44
3.4. System Architecture 45
3.4.1. EdgeAwareGNNLayer 45
3.4.2. PAE-GNN Architecture 46
3.4.3. Node-Only Baseline Architecture 47
3.4.4. Training Procedure 48
3.5. Ablation Study Design 48
3.6. Web Application Design 50
3.6.1. Backend Architecture 50
3.6.2. Frontend Architecture 51
3.7. System Modelling 52
3.7.1. Use Case Diagram 52
3.7.2. Activity Diagram 53
3.7.3. Sequence Diagram 55
3.8. Chapter Summary 57
CHAPTER FOUR RESULTS AND DISCUSSION 58
4.1. Introduction 58
4.2. System Implementation 58

4.3. PAE-GNN Training Convergence 59
4.4. PAE-GNN Classification Performance 61
4.4.1. Overall Performance 63
4.4.2. Per-Class Analysis 63
4.5. PAE-GNN Versus Node-Only Baseline 64
4.5.1. Baseline Training Behaviour 66
4.5.2. Performance Gap Analysis 66
4.6. Ablation Study Results 67
4.6.1. Interpretation of C1 vs C2: The Port Feature Contribution 69
4.6.2. The Role of Flow Statistics (C4) vs Port Features (C3) 70
4.6.3. The Well-Known Port Indicator (C5) 71
4.6.4. Minimal Configuration (C6) 71
4.7. Dashboard Demonstration 71
4.7.1. Forensic Analysis Mode 72
4.7.2. Live Monitoring Mode 73
4.8. Discussion 73
4.8.1. Performance in the Context of Prior Work 74
4.8.2. DDoS–DoS Confusion and Its Implications 75
4.8.3. The Port Feature Convergence Effect 75
4.8.4. Limitations 75
4.9. Chapter Summary 76
CHAPTER FIVE SUMMARY, CONCLUSIONS AND RECOMMENDATIONS 77
5.1. Introduction 77
5.2. Summary of the study 77
5.3. Conclusions 78
5.3.1. Acquisition and Preprocessing of the CSE-CIC-IDS2018 Dataset 78
5.3.2. Design and Implementation of the PAE-GNN Architecture 79
5.3.3. Training and Evaluation of the PAE-GNN on CSE-CIC-IDS2018 79
5.3.4. Comparison of PAE-GNN Performance Against the NodeOnlyGNN Baseline 79
5.3.5. Ablation Study to Quantify Port Feature Contributions 80
5.3.6. Deployment of the PAE-GNN as a Web-Based Monitoring Dashboard 80
5.4. Limitations of the Study 80
5.5. Recommendations for Future Work 81
5.5.1. Evaluation on Real-Time Streaming Graphs 81
5.5.2. Multi-Source Temporal Features for DDoS–DoS Disambiguation 81
5.5.3. Cross-Dataset Generalisation 82

5.5.4. Resolving the Train-Inference Node Feature Mismatch 82
5.6. CONCLUSION 82
References **84**

LIST OF FIGURES

FIGURES	TITLES OF FIGURES	PAGES
3.1	PAE-GNN Complete System Architecture	55
3.2	PAE-GNN Architecture - Two EdgeAwareGNNLayers followed by Port-Aware Edge Classifier	57
3.3	PAE-GNN Dashboard Architecture - FastAPI Backend and React Frontend	62
3.4	Use Case Diagram	64
3.5	Activity Diagram	66
3.6	Sequence Diagram	67
4.1	PAE-GNN Training and Validation Loss and Accuracy Curves	72
4.2	PAE-GNN Confusion Matrix	73
4.3	Per-Attack Type F1 Score Comparison — PAE-GNN vs Node-Only Baseline	76
4.4	NodeOnlyGNN Baseline Training and Validation Loss and Accuracy Curves	77
4.5	Ablation Study - Test Accuracy and Macro-F1 by Feature Configuration	79
4.6	Ablation Study - Per-Attack Type F1 Score Heatmap Across All Configurations	80
4.7	PAE-GNN Dashboard — Forensic Analysis Mode (Cytoscape.js COSE Layout)	83
4.8	PAE-GNN Dashboard — Live Map Monitoring Mode (Cytoscape.js Radial Layout)	84

LIST OF TABLES

TABLES	TITLES OF TABLES	PAGES
1.1	Objectives-Methodology Mapping Table	13
2.1	Comparative Summary of Related Works	32
2.2	Comparison of Existing NIDS Systems and PAE-GNN	36
3.1	Functional Requirements	43
3.2	Non-Functional Requirements	45
3.3	Hardware Requirements	47
3.4	Software Requirements	48
3.5	PAE-GNN Edge Feature Vector Specification	51
3.6	Ablation Study Feature Configurations	59
4.1	Implementation Environment	69
4.2	PAE-GNN Training Milestones	70
4.3	PAE-GNN Test Set Classification Report	72
4.4	NodeOnlyGNN Baseline Test Set Classification Report	75
4.5	Per-Class F1 Comparison - PAE-GNN vs NodeOnlyGNN Baseline	76
4.6	Ablation Study Results - All Feature Configurations	78
4.7	Per-Class F1 Scores Across Ablation Configurations	79

ABBREVIATIONS

AI	Artificial Intelligence
ANN	Artificial Neural Network
API	Application Programming Interface
CSE-CIC 2018	Canadian Institute for Cybersecurity Intrusion Detection Evaluation Dataset 2018
CNN	Convolutional Neural Network
DoS	Denial of Service
DDoS	Distributed Denial of Service
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GNN	Graph Neural Network
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IANA	Internet Assigned Numbers Authority
IP	Internet Protocol
IDS	Intrusion Detection System
IPS	Intrusion Prevention System
LSTM	Long Short-Term Memory
ML	Machine Learning
MPNN	Message Passing Neural Network
NIDS	Network-Based Intrusion Detection System
PAE-GNN	Port-Aware Edge-Feature Enhanced Graph Neural Network
REST	Representational State Transfer
SOC	Security Operations Centre
SGD	Stochastic Gradient Descent

SIEM	Security Information and Event Management
SMOTE	Synthetic Minority Over-Sampling Technique
SSH	Secure Shell
SVM	Support Vector Machine
TCP	Transmission Control Protocol

ABSTRACT

The proliferation of sophisticated cyber threats against networked enterprise infrastructure has exposed fundamental limitations in conventional intrusion detection approaches, which model network flows as independent tabular records and thereby discard the relational topology critical for identifying coordinated attack patterns. This paper has proposed and comprehensively validated Port-Aware Edge-Feature Enhanced Graph Neural Network (PAE-GNN), a novel graph neural network, for building directed, per-edge-attributed graphs from the CSE-CIC-IDS2018 dataset. Nodes in the graphs correspond to host IP addresses, and the edges encode communication flows between the hosts. Each edge is associated with fourteen features, of which an embedding explicitly encoding the destination port is added as one feature; the PAE-GNN uses edge-conditioned message passing in which per-flow port embeddings are used to condition the neighbourhood aggregation process. An ablation study was conducted across six feature configurations to quantify the contribution of port-aware features to classification performance. The trained model was deployed within a web-based monitoring dashboard built on a FastAPI backend and React frontend with Cytoscape.js graph visualisation, demonstrating end-to-end integration of graph-learning-based intrusion detection within an accessible operational interface. Results are reported in terms of accuracy, precision, recall, F1-score, and false positive rate. The findings confirm that edge-aware, port-informed graph representations offer a principled and practically deployable advance over tabular and node-centric baselines for network intrusion detection.

Keywords: Graph Neural Networks, Network Intrusion Detection, Edge-Conditioned Convolution, Port-Aware Features, CSE-CIC-IDS2018, Anomaly Detection, Deep Learning

CHAPTER ONE
INTRODUCTION

1.1. Background Information

The proliferation of networked computing infrastructure in enterprise, government, and other critical services environments has produced an exponentially increasing attack surface for malicious activity, which continues to create sustained demand for automated, intelligent systems that can identify intrusions in real-time in high-bandwidth network traffic (Ferrag et al., 2020). Today's networks produce high volumes of packet flow at such a scale and level of complexity that it simply cannot be studied manually; therefore, machine learning techniques that can identify attack signatures across both legacy threats and previously unseen variants are required. Signature-based Intrusion Detection Systems (IDS) are still the most prevalent to date, but as a reactive approach cannot identify zero-day signatures (Thakkar & Lohiya, 2021).

ML-based intrusion detection techniques tackle this zero-day issue by deriving an understanding of either normal or malicious traffic from labelled training data. Supervised classifiers such as Random Forests, SVMs and variants of gradient-boosted trees have achieved strong benchmark results on various datasets, notably the CSE-CIC-IDS2018 corpus published by Sharafaldin et al. (2018), and using ad-hoc feature extraction from per-flow traffic characteristics. They confirmed the viability of a data-driven approach to intrusion detection and continue to serve as strong baselines in the literature (Ferrag et al., 2020).

Non-architectural enhancements have been used to push detection capabilities further. For example, some techniques apply detection using raw or only lightly pre-processed traffic. CNNs then characterize flow records as matrices with learned features being captured by filters that focus on local interactions of elements within the matrices, while LSTMs take advantage of the temporal dependencies present between nearby flows in the traffic stream to characterize recurring patterns spanning multiple connections (Ferrag et al., 2020). Autoencoder architectures serve for unsupervised anomaly detection by identifying flows which depart significantly from the trained normal behaviour, in order to detect malicious connections without having any labelled attack examples. However, in spite of these architectural innovations, all tabular and sequence-based methods have a common

architectural weakness: they treat each flow independently, thus losing the host-to-host relational graph (the topology of attack strategies) through which collaborative attacks emerge (Thakkar & Lohiya, 2021).

Graph neural networks (GNNs) allow deep learning to be applied to graph-structured data using iterative neighbourhood aggregation, giving models the ability to learn immediate per-node feature content and multi-hop relational structure in parallel (Zhou et al., 2020). For network intrusion detection, at each time step IP addresses are naturally mapped to graph nodes, communication flows to directed edges, and an attributed graph is constructed where the overall structure provides a high-level view of the communication behaviour between hosts. The Graph Convolutional Network (GCN) model by Kipf and Welling (2017), along with its various extensions (e.g. GraphSAGE, GAT) has performed well on a suite of relational learning benchmarks, making GNNs the paradigm of choice for relational structure prediction problems.

Studies applying GNNs to intrusion detection, including E-GraphSAGE by Lo et al. (2022), have demonstrated that architectures incorporating edge features achieve significantly higher detection accuracy than node-centric or tabular baselines. However, a systematic gap persists across the existing GNN-based IDS literature: the destination port number, which directly identifies the targeted network service and is among the most diagnostically informative features for classifying port-specific attacks such as brute-force attempts on Secure Shell (SSH) and File Transfer Protocol (FTP) ports and web exploits on Hypertext Transfer Protocol (HTTP) and HTTPS ports, remains absent from current architectures as an explicitly embedded representation (Bilot et al., 2023). This study developed the Port-Aware Edge-Feature Enhanced Graph Neural Network (PAE-GNN), which addresses this gap by incorporating dedicated destination port embeddings within an edge-conditioned message-passing framework trained and evaluated on the CSE-CIC-IDS2018 dataset.

1.2. Statement of the Problem

Signature-based intrusion detection systems cannot detect novel attack variants that fall outside their rule databases, and machine learning systems operating on tabular flow records discard the inter-host relational topology through which coordinated attacks such as DDoS flooding, lateral movement, and botnet activity manifest as graph-level structures invisible to per-flow classifiers (Ferrag et al., 2020). Even the GNN-based approaches that partially address this representational gap predominantly omit destination port information from their

feature representations, reducing their capacity to discriminate between attack classes targeting specific network services, and the literature is further characterised by a predominance of binary rather than multi-class detection formulations that are insufficiently granular for operational incident response (Bilot et al., 2023).

The practical consequence is that security operations teams are left with either no detection against novel attacks or an undifferentiated malicious indicator that cannot inform the specific countermeasure required — whether traffic rate-limiting for DDoS, account lockout for brute-force, or application patching for web exploits — a compounding research and operational gap that the edge-aware, port-informed, multi-class PAE-GNN developed in this study directly addresses (Lo et al., 2022; Sharafaldin et al., 2018). Specifically, no published GNN-based IDS architecture has systematically evaluated the contribution of explicit destination port embeddings to multi-class classification performance, leaving the practical value of port-aware edge features for operational, service-differentiated intrusion detection empirically untested.

1.3. Aims and Objectives of the Study

The aim of this project is to establish a Port-Aware Edge-Feature Enhanced Graph Neural Network (PAE-GNN) based network intrusion detection model on CSE-CIC-IDS2018 dataset on a web-based monitoring dashboard with the following specific goals:

i. Data preprocessing of CSE-CIC-IDS2018 dataset, which would include data cleaning, feature normalisation, class imbalance balancing through the use of SMOTE, and labelling for graphs.

ii. To build a directed, edge-attributed graph representations of network traffic and map the IP host addresses in networks to graph nodes and each communication flow to a graph edge with fourteen features such as destination port, protocol and flow level statistic.

iii. To design and train the PAE-GNN architecture with destination port embeddings and edge-conditioned message passing, using PyTorch and PyTorch Geometric;

iv. In order to test the trained model by standard measurement, such as accuracy, precision, recall and F1-score, and also false positive rate.

v. To perform ablation experiment on port-aware and other edge feature group respectively to evaluate their effect on the classification.

vi. To deploy the trained model on the web based dashboard, which is based on API FastAPI backend and React on the frontend to visualise the traffics and anomalies.

1.4. Methodology

The employed research method in this study is a hybrid of an applied design and an evaluatory investigation. The applied design aspect refers to the design and construction of a novel GNN based intrusion detection system; while the evaluatory aspect involves the systematic empirical evaluation of its performance in terms of baseline comparison and ablation study. The combination is suitable because the research questions demands an artefact as well as an empirical controlled experimentation. The method was realised in the context of a six-phase project, where each phase corresponds to one of the research objectives listed in Section 1.3. The relation between each objective, appropriate method/tools used and output is summarized in Table 1.1.

Table 1.1: Objectives-Methodology Mapping Table

#	Objective	Method / Technique	Tools Used	Expected Output
i.	Preprocess the CSE-CIC-IDS2018 dataset for graph-based learning	Data cleaning, feature normalisation, SMOTE oversampling, label encoding	Python, Pandas, scikit-learn, imbalanced-learn	Balanced, normalised feature matrix with multi-class attack labels
ii.	Construct attributed graph representations of network traffic flows	IP-based graph construction; nodes = hosts, edges = flows with 14 edge features	Python, PyTorch Geometric	Directed attributed graph of network communication
iii.	Develop the PAE-GNN with port-aware edge-conditioned message passing	Edge-conditioned convolution, destination-port embeddings, multi-layer GNN	PyTorch, PyTorch Geometric	Trained PAE-GNN model capable of edge-level traffic classification

#	Objective	Method / Technique	Tools Used	Expected Output
iv.	Evaluate PAE-GNN classification performance	Accuracy, Precision, Recall, F1-Score, False Positive Rate	scikit-learn, Matplotlib, Seaborn	Quantitative evaluation report and confusion matrix
v.	Conduct ablation study to assess contribution of edge features	Systematic removal of feature groups across six configurations	PyTorch, scikit-learn	Feature importance ranking confirming port-aware contribution
vi.	Deploy PAE-GNN as a web-based intrusion detection dashboard	RESTful API backend, React frontend, real-time graph visualisation	FastAPI, React, Tailwind CSS, Cytoscape.js	Functional dashboard for traffic monitoring and anomaly visualisation

The process covers four steps, data collection and preprocessing, graph construction, model training, evaluation and deployment. The Stage

i oversampling with SMOTE is used to alleviate the very unbalanced class distribution of CSE-CIC-IDS2018 dataset; the Stage

ii graph construction is used to convert the preprocessed flow records into the form of attributed graph which is needed by PAE-GNN. The Stage

iii training exploits the edge-conditioned convolution primitives of PyTorch Geometric; the Stage v ablation study confirms the empirical soundness of port-aware architectural choices.

1.5. Significance of the Study

The significance of this study is reflected in the following benefits to network security research and practice:

- It benefits network security practitioners and organisations by addressing the limitations of existing automated intrusion detection systems, which either rely on static signature databases that cannot detect novel threats or employ tabular machine learning models that discard the relational topology through which coordinated attacks manifest (Thakkar & Lohiya, 2021).
- Thus by building an edge-aware, port-informed graph neural network framework, it would allow for more precise and finer detailed detection of network attacks and corresponding defensive action in a timely manner.
- It helps advance the use of graph-structured deep learning in cybersecurity, in particular for enterprise network monitoring, and motivates the transition of novel machine learning architectures into standard security tooling (Bilot et al., 2023).

Moreover, the deployment of the PAE-GNN within a web-based dashboard ensures that the system is accessible to security analysts, network administrators, and researchers without requiring direct interaction with the underlying model pipeline, enabling real-time diagnosis and visualisation of anomalous traffic patterns from any network-connected client device (Lo et al., 2022). This lowers the barrier to adoption of graph-learning-based detection in operational Security Operations Centres (SOCs). In addition to its practical significance, the study contributes academically by providing a documented evaluation of a novel GNN architecture on the CSE-CIC-IDS2018 benchmark, advancing understanding of edge-conditioned message passing for intrusion detection and providing a reference implementation that can be extended by future researchers investigating port-aware or multi-class graph-based anomaly detection systems.

1.6. Project Outline

This study consists of five chapters: the first chapter contains the background of the study and its problem statement, the aims and objectives, the methodology, and the study's significance; the second chapter presents the literature review covering relevant concepts, theoretical framework, related works, existing systems, identified gaps, and a summary; the third chapter presents the research methodology including the mixed research design, requirement analysis, data preprocessing and graph construction, PAE-GNN architecture, ablation study design, web application design, and UML system models; the fourth chapter presents the results and discussion including model training outcomes, classification

performance evaluation, baseline comparison, ablation study findings, and dashboard demonstration; and the fifth chapter provides the summary, limitations, recommendations, and conclusion of the study.

CHAPTER TWO
LITERATURE REVIEW

2.1. Preamble

This chapter starts off with a systematic review of the literature concerning the design and evaluation of the Port-Aware Edge-Feature Enhanced Graph Neural Network (PAE-GNN). The literature review is structured in such a way as to lead from the fundamental concepts of the domain of the technology, through theoretical and conceptual approaches, a critical review of related work, a review of operational systems, culminating in a review of the identified research gaps that the PAE-GNN attempts to fill (Bilot et al., 2023).

Primary sources examined in this chapter were primarily peer reviewed articles of between 2018 and 2025, obtained via IEEE Xplore, the ACM Digital Library, Google Scholar, and arXiv; preprints from arXiv were only included where their peer-reviewed equivalent was unavailable. Search queries included combinations of "intrusion detection", "graph neural network", "network anomaly detection", "edge-conditioned convolution", "CSE-CIC-IDS2018", and "port-aware classification". When multiple papers were authored to address the same phenomenon, preference was given to the most recent and most cited and well validated work (Sharafaldin et al., 2018). A handful of papers published prior to 2000 are intentionally cited as principal references for the underlying concepts, rather than as contemporary factual support: Postel (1981) is the initial standard metes and bounds for ports, while Roesch (1999) and Paxson (1999) are seminal articles discussing the origins of the Snort and Bro/Zeek intrusion detection frameworks which are still in widespread operational use.

The organization of this chapter mirrors the dependencies in technical contributions: Network security and intrusion detection introduces the problem space, machine learning techniques serve as a framework for the graph-based methods, and graph theory with graph neural network architectures supplies the representational and computational language to lay the foundation for the PAE-GNN design. The various systems based on GNNs and existing deployed tools are described in a comparative manner with the PAE-GNN framework established in order to identify the specific research gaps this dissertation addresses (Zhou et al., 2020).

2.2. Review of Relevant Concepts

This section introduces the core concepts on which the PAE-GNN is founded. Three principal knowledge domains are examined in sequence: network security and intrusion detection systems, which establish the problem context and detection paradigms; machine learning in cybersecurity, which surveys the methods against which graph-based approaches are contrasted; and graph theory together with graph neural networks, which provides the representational and architectural foundations of the system. Each subsection introduces the relevant concepts before examining their specific implications for the present study.

2.2.1. Network Security and Intrusion Detection Systems

Network security is defined as the collection of policies, hardware controls and monitoring systems used to ensure the integrity, confidentiality and availability of data being transmitted across computer networks (Kwon et al., 2019). As networked systems are now integral to organisations' processes, the corresponding increase in reliance on this infrastructure has been met with an escalating volume of attacks seeking to compromise these environments. Attacks often go undetected due to the plethora of possible vulnerabilities across the network stack from the physical layer through to applications-layer code injection and man-in-the-middle attack, and subsequent damage includes data theft, asset destruction, service interruption, financial fraud and damage to reputation. Network intrusion detection seeks to defend against these attacks by creating a monitoring system that is capable of identifying malicious activity occurring in network traffic streams either as the attack occurs or via analysis of stored data flow records (Ferrag et al., 2020).

An intrusion detection system (IDS) is a monitoring infrastructure set up with the aim of reporting malpractices or rule violations that occur within a network environment or on a specific host. Conventionally, an IDS platform is divided in accordance with two axes; namely the detection mechanism and the hosting platform. Regarding the classification with respect to the detection mechanism, IDS are based on three different detection approaches: signature-based, anomaly-based and hybrid systems. Signature-based systems, also referred to as misuse detection systems, use a database of attack signatures against which information flows are compared; alert is generated when a match is found with the existing signatures (Ferrag et al., 2020). Anomaly-based systems establish a statistical profile of normal activities with the help of a mixture of models and generate an alert when a deviation from

the established profile occurs; theoretically, these systems are able to detect previously unknown and zero-day attack variants (Thakkar & Lohiya, 2021).

Regarding placement of the IDS system, the deployed system can be a Network-based IDS (NIDS), observing traffic at a router or switch chokepoints, or a Host-based IDS (HIDS), which observes activity on a single machine (Ferrag et al., 2020). Hybrid platforms exist which combine both systems and, in many implementations, the two paradigms; these systems combine signature matching for known threats with an anomaly model running in parallel for unknowns. Regardless of the detection paradigm, all traditional IDS systems that operate with feature records on each network flow have a fundamental problem: because every flow is treated as an independent sample, relational connections between communicating hosts are broken, and the system cannot discover attack patterns in the form of a topological pattern extending across multiple flows (Lo et al., 2022). This problem motivates the graph-structured approach in PAE-GNN.

Network-based IDS solutions differ in whether they operate in passive (monitor-only) or in active inline mode, where they serve as Intrusion Prevention Systems (IPS) with the ability to dropped or shape malicious traffic. The dataset used for analysis here, the CSE-CIC-IDS2018, was created in an environment meant to mirror that of a monitored enterprise network, with attributions for legitimate traffic and a diverse set of attack types across a five-day window (Sharafaldin et al., 2018). All the types of attacks present in the dataset - DDoS flooding, DoS variants, botnet command-and-control, brute-force authentication, and web application exploits - are also those that a pragmatic NIDS would need to be able to provide protections for.

2.2.2. Machine Learning in Cybersecurity

Literature on machine learning for network intrusion detection has proliferated across classical supervised learning, ensemble and deep learning models. The primary justification for any machine learning approach is that a learned model can generalise from available training data to previously unseen examples, which avoids the zero-day detection problem with signature detectors and also eliminates the time-consuming task of manually engineering detection rules for each new class of attack (Thakkar & Lohiya, 2021). A significant amount of work has established that supervised classifiers trained on labelled network flow records can perform well on benchmark datasets, but the representation of those records is subject to limits in the things the models can learn.

Although frequently applied classifiers for intrusion detection are Random Forest classifiers, given its ability to safely handle noisy features, that do not have to be filtered out due to the implicit feature selection, via use of the information theoretic based splitting criterion, and that do not tend to overfit as an ensemble model via averaging, their results have been good and comparable to other ML approaches, due to the good generalisation ability of its learned models, and fast inference times, that also allow implementation in latency-sensitive application environments (Ferrag et al., 2020). Support vector machine classifiers with radial basis function kernels, that perform well on binary classification tasks, with difficulties when scaling in size, to larger high dimensional datasets, are also amongst classical ML classifiers. These establish the tabular ML models used as reference in the intrusion detection setting that are employed in GNN based approaches studies in the literature, with GNN being classified as superior, when data makes use of a graph structure (Bilot et al., 2023).

Deep Learning (DL) architectures. Convolutional Neural Networks (CNN) have been employed to traffic data by representing flow records as 2-dimensional matrices in order to allow the local feature interactions to be learned by the network through the application of the kernel-based convolution operation. Long Short Term Memory (LSTM) networks have been used to model the interactions between traffic features over time, learning patterns that extend across subsequent flows rather than within a flow itself. Lastly, autoencoder architectures have been used for unsupervised anomaly detection by learning the low-dimensional representations of benign traffic with the input subsequently identified as anomalous if the reconstruction error exceeds a predefined threshold. All tabular and sequence models are structurally limited however to model network traffic without taking into account the relation topology of hosts communication with each other making them oblivious to graph-level signatures of 2+ hosts coordinated attack (Lo et al., 2022).

Another commonly encountered issue in applying machine learning to network intrusion detection is class imbalance, as most network traffic datasets both publicly available and industry benchmark contain many more benign flows than attack flows for most attack classes. CSE-CIC-IDS2018 exhibits this property, where the attack classes comprise a vanishing proportion of the total flows (Sharafaldin, et al., 2018). The presence of class imbalance in the training set causes standard classifiers to optimise for majority-class accuracy, leading to models that prevaricate every input as benign traffic with high accuracy but fail also to classify attack classes at all. One of the most popular mitigation strategies for

class imbalance in this domain is the Synthetic Minority Over-sampling Technique (SMOTE), which interpolates between minority-class samples in feature space in order to create duplicate attack instances (Thakkar & Lohiya, 2021). To remedy the imbalance in the training set, SMOTE is applied during preprocessing in this work.

2.2.3. Graph Theory and Graph Neural Networks

Graph theory is the mathematical language in which systems of interacting entities can be modeled and reasoned about. A graph G = (V, E) consists of a vertex set V representing entities, and an edge set E containing pairs of vertices with specified relationships between them (Zhou et al., 2020). Graphs may be directed or undirected, simple or multigraph (in which pairwise edges are allowed between vertices). For modelling network traffic, directed graphs are a natural fit, as network flows will emanate from an Internet Protocol (IP) address representing a source node, towards one representing a destination node. The matrix A encodes the connectivity between nodes of the graph, where entry aij is one if and only if there exists an edge from node i to node j, and the degree matrix D tallies the number of edges incident on each node.

Nodes, edges or both may be attributed in a graph as well. An edge-attributed graph assigns a feature vector eij to each directed edge (vi, vj), representing the properties of the corresponding flow. For network intrusion detection, an edge feature vector encodes flow-level statistics, protocol identifiers, packet count, byte rate, inter-arrival time distributions, flow flags, and, of particular importance to the PAE-GNN, the destination port number reflecting the network service under attack. An edge-attributed directed graph representation captures the relational structure and the flow-wise diagnostic information needed for high-precision intrusion detection, but in a form that conventional tabular data structures cannot (Lo et al., 2022). The PAE-GNN graph-building schema maps each distinct source IP to a source node, each destination IP to a destination node, and each flow record to a directed edge with attribute vectors.

Graph neural networks (GNNs) are a family of deep learning models that operate on graph-structured data by iteratively aggregating feature information from neighbourhoods to learn representations which encode both the content of local features as well as multi-hop neighbourhoods (Zhou et al., 2020). The common underlying operation for most GNN architectures is message passing, where each node updates its feature content in a layer by aggregating an attribute transformation over its neighbouring nodes. In doing so, after k

rounds of message passing, each node's feature content contains information from a node'sk-hop neighbourhood. Kipf and Welling (2017) modelled a single step of message passing as a spectral graph convolution, with a symmetrically normalised Laplacian smoother. Inductive extension of the model to unseen graphs was achieved by sampling fixed-sized local neighbourhoods of nodes, and was named GraphSAGE. Attention coefficients can be computed over the neighbourhood to selectively emphasise each neighbour's features depending on their interaction with the central node's features (Velickovic et al., 2018). The application of these architectures to network intrusion detection has been reviewed recently (Zhong et al., 2024).

The message-passing architecture has been extended to incorporate edge features within the aggregation operation itself. In edge-conditioned convolution, each per-edge weight matrix is generated by a learned function of the edge feature vector, all of which are learned together to condition the flow of information according to the specific relational properties of a particular edge in the graph, rather than applying the same filter to each edge equally (Simonovsky & Komodakis, 2017). The ability to do this is directly applicable to network intrusion detection, because the labels of the fourteen edge features in the PAE-GNN include the destination port number which reveals the targeted service at the destination host, and at the same time provides very strong diagnostic clues as to the attack class for a particular network flow.

2.3. Theoretical and Conceptual Framework

The proposed PAE-GNN is based on the theoretical foundation of a three-pillar framework - graph-theoretic network modelling, message-passing neural computation, and port-aware service semantics, explaining the theoretical assumptions, design architecture, and conceptual association behind the system design. Having a well-defined theoretical foundation enables us to separate theory-driven design decisions from empirically-framed engineering decisions, and serve as the basis of experiments which, as Gilmer et al. (2017) state, "interpret experimental results within the confines of the architecture's theoretical assumptions".

The core theoretical premise is: intrusion behaviours on a network are graphtopological and cannot be completely characterised by per-flow statistics alone. There is empirical evidence to support this assumption for coordinated attack structures in the nature of their topological signatures: DDoS: Topologically concentrated in-degree at targeted nodes; Scanning:

Behavioural scanning systems also exhibit a high out-degree with short flow durations; Botnets: regular fan-in (distributed bots–controller communications) of constant bandwidth (Bilot et al., 2023). A model that works only with the individual flow feature vectors and does not take into account the graph context of each flow has structurally limited knowledge of the communication structure, and therefore the graph-based attack pattern.

The second theoretical foundation is the Message Passing Neural Network (MPNN) framework formalised by Gilmer et al. (2017), which encapsulates a large family of popular GNN models in terms of three functions: a message passing function which produces a message vector for each directed edge given the features of this edge's source and destination nodes, an aggregation function for combining the set of message vectors received at each node, and an update function for combining the set of messages and the previous node features to produce the new node features. Our PAE-GNN model uses an edge-conditioned message function which includes the learned port embedding to produce semantically meaningful messages from each flow to be conveyed to the target node. This message function tests whether port-aware messaging results in richer intrusion-specific node representations.

The third pillar port-aware service semantics – supplies the domain justification for elevating destination port features to being a first-class feature. In the TCP/IP Protocol Stack the destination port number defines the precise network service being targeted by the flow. These are defined by the Internet Assigned Numbers Authority – for example Port 22 indicates SSH, port 80 HTTP, port 443 HTTPS etc (Postel 1981). Attack traffic clusters around certain port ranges with brute-force ports 21, 22 and 3389, web application ports 80 and 443, and certain DDoS amplification ports being used with UDP based services. A model that performs port embedding learns a representation space in which semantically related services (and hence related attack domains) will be nearby in embedding space providing generalisation between instances of port-specific attack classes. This three-pillar approach is realised in the graph construction methodology and the edge-conditioned architecture in Chapter Three.

2.4. Review of Related Works

This section provides a critical review of the existing body of literature for the most relevant papers to the PAE-GNN which have used graph neural networks or graph-structured representations to solve network intrusion detection and anomaly detection problems. The

individual studies are discussed in detail, focusing on method, data, findings, and the particular strengths and weaknesses of each study compared to PAE-GNN. All the reviewed studies are summarized in Table 2.1. The presentation is presented from the most architectural proximity to more contextually relevant works.

2.4.1. E-GraphSAGE: A Graph Neural Network Based Intrusion Detection System

Lo et al. (2022) introduced E-GraphSAGE, an extension of the GraphSAGE inductive neighbourhood sampling framework by adding edge features to the neighbourhood aggregation process for network intrusion detection. The model was built by extending the neighbourhood representations with the edge feature vectors at each layer of message passing, allowing it to learn from properties of individual flows, in addition to topological structure of the host communication graph. E-GraphSAGE was tested on the CICIDS2017 dataset and was shown to outperform node-only GNN baselines used in the same study with F1 scores higher than 0.97 on most attack classes. The architecture is a substantial improvement over previous GNN-based IDS systems that only considered flow attributes as nodes.

The closest architectural ancestor of the PAE-GNN is E-GraphSAGE, and the edge-aware message-passing scheme used in this study was inspired by the E-GraphSAGE design. There are, however, two drawbacks as opposed to the PAE-GNN approach. First, E-GraphSAGE does not use explicit destination port embeddings, and (when available) port information is simply treated as a raw numeric feature, unlike other continuous flow features the model receives. This will prevent the model from learning the service-semantic structure of port space or generalise across port specific patterns of attack that could be different at each port but are semantically related. Second, the evaluation of E-GraphSAGE is limited to binary classification on CICIDS2017, on which they did not provide proof of multi-class discrimination ability, which is necessary for a practical attack categorisation. The PAE-GNN directly expands the paradigm of edge-aware introduced by Lo et al. (2022) by adding a dedicated port embedding structure and testing on the recently proposed CSE-CIC-IDS2018 benchmark.

2.4.2. Edge-Directed Graph Multi-Head Attention Networks for Network Intrusion Detection

Li et al. (2023) proposed Edge-Directed Graph Multi-Head Attention Networks (EDGMAT) for network intrusion detection, motivated by the observation that existing GNN-based

detectors such as E-GraphSAGE aggregate neighbouring nodes with equal weighting and thereby underuse both edge features and the directionality of network flows. EDGMAT constructs a directed graph in which network flows are modelled as edges carrying their own feature vectors, and it introduces a multi-head attention mechanism that combines these edge features with learned attention weights so that neighbouring nodes and their linked edges contribute unequally during aggregation according to their similarity. Compared to four state-of-the-art baseline approaches on four recent NIDS benchmark datasets, in the multi-class detection setting, EDGMAT is claimed to demonstrate weighted F1-score improvements by a substantial margin. The results indicate that the attention-weighted, edgefeatured, direction-aware graph representations effectively separate multiple attack types, and are not simply an improvement to binary detection.

While EDGMAT advances the use of edge features and attention in intrusion detection, it does not treat the destination port as a semantically privileged feature warranting a dedicated learned embedding; port information, where present, is handled as one undifferentiated edge feature among others rather than being mapped into a continuous service-aware representation space. As a consequence, the model does not explicitly learn the service-level relationships between ports that distinguish attacks targeting specific network services. The authors additionally note that the multi-head attention mechanism incurs non-trivial computational and memory overhead, identifying the reduction of training time and GPU usage as an avenue for future work. The PAE-GNN retains the edge-featured, direction-aware, multi-class design philosophy shared with EDGMAT while addressing the port-representation gap directly through a dedicated destination port embedding incorporated within its edge-conditioned message-passing process.

2.4.3. A Survey of GNNs for Network Intrusion Detection

Bilot et al. (2023) published a systematic survey on network intrusion detection with GNNs. The survey analyzed more than 40 published papers discussing architecture types, datasets, performance metrics, and deployment characteristics. The authors of the survey claim that GNN-based methods can consistently outperform tabular machine learning baselines by three to 12 percentage points across benchmark datasets, with the likely reason being models' ability to grasp topological attack signatures invisible to per-flow classifiers. Two common issues were found in the analyzed literature: binary classification formulations,

which probably lack the granularity required for operational incident response, and the lack of use of edge features, especially port, in most of the analyzed architectures.

The survey by Bilot et al. (2023) provides direct empirical justification for two core design choices of the PAE-GNN. The identification of binary-classification predominance as a limitation motivates the multi-class formulation of this study. The observation that port features are systematically underutilised in GNN-based IDS architectures directly motivates the port embedding mechanism that constitutes the primary novel contribution of the PAE-GNN. As a survey, the work does not propose a novel architecture, and it explicitly calls for future research to address the identified gaps. The PAE-GNN can be understood as a direct response to this call, operationalising the specific architectural improvements that Bilot et al. (2023) identify as absent from the literature.

2.4.4. Anomal-E: Self-Supervised Edge-Based GNN for Anomaly Detection

Caville et al. (2022) proposed Anomal-E, a self-supervised GNN-based anomaly detection system that learns edge representations from unlabelled network traffic using contrastive learning objectives. The system constructs an edge-attributed graph and trains a GNN encoder to maximise the similarity between representations of edges that appear in close temporal proximity while minimising similarity between randomly sampled edge pairs. At inference, edges whose representations are distant from the learned normal distribution receive high anomaly scores. Anomal-E was evaluated on the ToN-IoT and CICIDS2017 datasets and demonstrated competitive anomaly scoring performance without requiring labelled attack examples during training, addressing a significant practical constraint in enterprise IDS deployment where labelled attack traffic is scarce.

The Anomal-E architecture demonstrates that edge-level graph representations are well-suited to the intrusion detection task and that contrastive self-supervised objectives can produce useful edge embeddings from unlabelled traffic. The edge encoding design of Anomal-E informs the encoder architecture of the PAE-GNN, particularly the approach of treating flows as first-class graph entities with rich feature representations. However, the self-supervised formulation of Anomal-E produces anomaly scores rather than categorical attack class predictions, making it complementary to rather than a substitute for supervised multi-class architectures. In operational settings that require specific attack categorisation to inform differentiated incident response, a supervised model such as the PAE-GNN is

necessary. The two approaches are therefore not in competition but address different segments of the IDS capability spectrum.

2.4.5. Heterogeneous Graph Neural Network for Intrusion Detection

Deng et al. (2022) used a heterogeneous graph representation of CSE-CIC-IDS2018 to learn cross protocol communication dynamics. Different types of edges were used for TCP flows, UDP flows and ICMP flows in the same graph to allow the model to learn protocol-specific communication dynamics in the same graph. Because separate learnable parameters for TCP flows, UDP flows and ICMP flows as semantically different edge types were used in the heterogeneous GNN, we would expect the model to learn attack characteristics that only exist on specific protocol behaviors. The heterogeneous GNN has better detection accuracy on protocol specific attack classes than the homogeneous graph baselines that treat all edge types the same. This seems to support the hypothesis that encoding domain specific categorical features as separate structural components or embeddings could improve detection performance.

(Deng et al., 2022) is directly relevant to the PAE-GNN in that they demonstrate the importance of encoding categorical network-layer identifiers (e.g. Protocol type) as structurally meaningful signals instead of raw decimally-encoded numbers. The PAE-GNN is inspired by this work in that it applies this insight to the service layer: it encodes the destination port number (through which an implementation can access a more fine-grained specification of the service in question than protocol type alone). In particular, a flow to port 80 and a flow to port 443 are both TCP flows; a naïve protocol-aware model would not distinguish between them, while a port-aware model would be differentiated as HTTP or HTTPS flows, respectively. This distinction is relevant to attacks that are targeted at one or the other. Ultimately, the PAE-GNN is a conceptual deepening of the insight conveyed by (Deng et al., 2022), in that it extends this insight from protocol awareness to service awareness.

2.4.6. IGNNITION: GNN-Based Traffic Classification with NetFlow Features

Pujol-Perich et al. (2022) investigated GNN-based network traffic classification using features derived from NetFlow records across multiple datasets including CICIDS2017 and UNSW-NB15, demonstrating competitive F1 scores and cross-dataset generalisation. The study used the IGNNITION framework to construct and evaluate GNN models for traffic classification, reporting that models trained on one network environment could achieve non-

trivial performance on traffic captured from a different environment. This cross-dataset generalisation property is practically important for deployable IDS, where the target deployment network differs from the training environment. The study demonstrated GNN applicability to a multi-dataset evaluation paradigm that is more representative of real-world deployment conditions than single-dataset benchmarks.

The main limitation directly relevant to the PAE-GNN concerns the missing destination port feature used by Pujol-Perich et al. (2022). This leaves the port-specific detection ability untested in a paper that otherwise demonstrates significant evidence for GNN traffic classification on the generalisability front. Also, the absence of port features in a multi-dataset setting also means the performance differences reported cannot be attributed to or checked in the context of port-aware models. The PAE-GNN in contrast explicitly models destination port embeddings and assesses their impact in an ablation study, which is a feature not demonstrated in Pujol-Perich et al. (2022).

2.4.7. DÏoT: Federated GNN Anomaly Detection for IoT

Nguyen et al. (2021) performed a study on federated learning with GNN components for distributed anomaly detection in IoT environments, using the UNSW-NB15 dataset. The federated model training distribution across several network edges allows the aggregation of locally trained model updates into a global model without having to transfer the raw traffic datasets, thus maintaining individual network environment privacy while not sacrificing classification accuracy compared to centralized training. This demonstrates that detection using GNN models is feasible in privacy-preserving distributed deployment scenarios, as it would be the case in large enterprise and multi-tenant network environments.

Nguyen et al. (2021) are mainly relevant to the PAE-GNN as confirmatory rather than architecture. Based on the results of Nguyen et al. (2021), it has been demonstrated that GNN architecture can be successfully leveraged for IDS and trained to correspond to distributed constraints without de minimus impact. It must be noted that by virtue of their focus on federated architecture and IoT, Nguyen et al. (2021) are outside of the set against which the PAE-GNN can be compared – which is designed for enterprise Network Intrusion Detection System (NIDS) trained with the CSE-CIC-IDS2018 data. However, if the PAE-GNN were to be extended to include federated architectures as alluded to in Chapter Five and beyond, Nguyen et al. (2021) methodology might prove instructive.

2.4.8. GCN with Attention for Botnet Detection

Tang et al. (2022) introduced an attention-based GCN for botnet traffic detection on the CICIDS2017 dataset, resulting in a detection rate of 99.2% for the class of botnet attack, out of 11 attack classes. The high classification result for the attacker class that exhibit the most topologically distinct traffic pattern, which is the class of botnet command-and-control traffic, supports the discriminative potential of attention enhanced GCN models when the topology of the targeted attacker traffic class is more cohesive and consistent, providing strong learning conditions. The attention mechanism allows the GCN model to identify and attend to the most salient inter-host connections, effectively capturing the fan-in structure typically exhibited by the targeted attacker class.

The main drawback of this approach, as identified by Tang et al. (2022), is the singular focus on single-class evaluation, which provides a sound benchmark for performance on botnets, but does not test the ability of the model to generalize to other attack types or in the context of a multi-class classification task. A model that classifies botnet traffic with near-perfect accuracy may perform significantly worse on DDoS or brute-force attacks, which likely contain topologically very distinct signatures. In addition, the single-class approach yields no evidence for how many false positives would have been generated under the full distribution of benign traffic patterns. The PAE-GNN alleviates this problem by formulating the problem as a multi-class classifier from the outset, forcing the model to learn discriminative embeddings across multiple attack classes simultaneously.

2.4.9. Simplifying Graph Convolutional Networks for Intrusion Detection

Wu et al. (2019) proposed SGC, a Simplified Graph Convolutional Network, which reduces the number of the non-linear transformation layers in GCN to a single linear mapping, thereby drastically reducing the compute time and memory requirements while incorporating the neighborhood aggregation which has been a key factor of the success of the GCN architecture. The network was tested in network intrusion detection with the KDD Cup 99 benchmark dataset and the NSL-KDD subset, obtaining a classification accuracy of 96% and much lower inference latency than multi-layer GCNs. This study indicates that more simple GCN networks may be deployable with the latency range required for network monitoring, and that the more complex and expressively non-linear deep GCN architecture may not be necessary for detection problems where the decision boundary complexity is relatively low.

However, the primary limitation of Wu et al. (2019) applying to the Port-Aware Edge-enhanced Graph Neural Network (PAE-GNN) is the use of outdated benchmark datasets in their study, KDD Cup 99 and NSL-KDD, whose traffic patterns differ from modern attack techniques. As a result, the results obtained from their study may not generalize well across modern enterprise network environments. In addition, the SGC model is constructed solely with node features and does not utilize edge features, placing it in the node paradigm which the PAE-GNN seeks to extend upon. Nonetheless, the work still serves as a valid reference point for GCNs, particularly as a comparative lower-bound to evaluate the efficacy of edge-conditioned, port-aware graph structures on the CSE-CIC-IDS2018 dataset.

2.4.10. Toward Generating a New Intrusion Detection Dataset

Sharafaldin et al. (2018) created the CSE-CIC-IDS2018 dataset in a collaboration between the Communications Security Establishment Canada and the Canadian Institute for Cybersecurity. The dataset was created on a simulated enterprise network using the B-Profile behaviour-based traffic generator, which accurately models user behaviour over various applications and protocols for natural attack traffic simulation. Seven attack scenarios were executed over a span of multiple days including DDoS, DoS Hulk, DoS Golden Eye, brute-force authentication on FTP and SSH, botnet using Ares botnet, web application attack (SQL injection and cross-site scripting), and infiltration attack (Metasploit), for the attack categories. Over 80 flow-based features were extracted from the raw packet captures using the CICFlow Meter tool.

This tabular dataset fulfills a secondary role in the current paper, as a data source for training and evaluating PAE-GNN, and as a reference for feature extraction and attack scenario design for the benchmark. (2018) have provided benchmark evaluation results on the tabular feature representation using Random Forest and other classical machine learning classifiers, as a reference point for the performance enhancement possible with graph-structured techniques. The tabular ML evaluation also serves to demonstrate the class imbalance problems and feature engineering capabilities that the graph construction step and the SMOTE undersampling pipeline of the PAE-GNN are aiming to address.

2.4.11. A Comprehensive Study on GNN Architectures and Applications (Zhou et al., 2020)

Zhou et al. (2020) provided a comprehensive survey of graph neural network architectures and their applications across social networks, chemistry, biology, knowledge graphs, and computer vision. The survey identified message passing, spectral methods, and spatial methods as the three principal computational paradigms and traced the development of the field from the foundational GNN model of Scarselli et al. (2009) through the GCN, GraphSAGE, and GAT architectures. The survey established the general-purpose applicability of GNN architectures to relational reasoning tasks and provided a theoretical taxonomy within which specific architectures, including the edge-conditioned models relevant to the PAE-GNN, can be positioned.

The cybersecurity application domain is briefly surveyed by Zhou et al. (2020) but the edge-conditioned, port-aware detection task targeted by the PAE-GNN falls outside the scope of works reviewed at the time of publication. The survey's primary contribution to the present study is theoretical grounding: by establishing the expressiveness of message-passing GNNs and the conditions under which they can distinguish graph structures, the survey provides the theoretical basis for claiming that the PAE-GNN's edge-conditioned architecture is capable in principle of learning the topological signatures of distinct attack categories. The taxonomy of GNN architectures provided by Zhou et al. (2020) also informs the positioning of the PAE-GNN relative to GCN and GAT baselines in the evaluation.

Table 2.1: Comparative Summary of Related Works

Author(s)	Year	Dataset	Methodology	Key Results	Limitations	Relevance to PAE-GNN
Lo et al.	2022	CICIDS2017	E-GraphSAGE: edge-feature GraphSAGE for IDS	F1 > 0.97 on most attack classes	No port embeddings; binary classification only	Closest architectural predecessor; PAE-GNN adds explicit port encoding
Li et al.	2023	Four NIDS benchmarks	Multi-head attention over directed, edge-featured flow graphs (EDGMAT)	Higher weighted F1 than four baselines on multi-class detection	No dedicated destination port embedding; attention adds compute/memory overhead	Validates CSE-CIC-IDS2018 suitability; highlights edge-level gap
Bilot et al.	2023	Multiple (survey)	Systematic survey of GNN methods for NIDS	GNNs outperform tabular ML by 3–12%	Survey only; no novel architecture	Empirically justifies GNN-based approach; identifies port-feature gap
Caville et al.	2022	ToN-IoT, CICIDS2017	Anomal-E: self-supervised GNN with edge features	Strong anomaly scores without labels	Unsupervised; no multi-class discriminative power	Edge learning design informs PAE-GNN encoder
Deng et al.	2022	CSE-CIC-IDS2018	Heterogeneous graph with protocol-typed edges	Improved detection for protocol-specific attacks	Protocol type only; destination port not embedded	Protocol-aware design directly motivates port embedding

Author(s)	Year	Dataset	Methodology	Key Results	Limitations	Relevance to PAE-GNN
Pujol-Perich et al.	2022	CICIDS2017, UNSW-NB15	GNN with NetFlow-derived node features	Competitive F1; cross-dataset generalisation	Destination port excluded from feature set	Motivates port-aware feature inclusion across datasets
Nguyen et al.	2021	UNSW-NB15	Federated GNN for distributed IDS	Privacy-preserved accuracy near centralised baseline	Federated overhead; no port-aware features	Confirms GNN viability in IDS; deployment context relevant
Tang et al.	2022	CICIDS2017	GCN with attention for botnet detection	99.2% botnet detection accuracy	Single class; no multi-class generalisation	Motivates multi-class formulation in PAE-GNN
Sharafaldin et al.	2018	CSE-CIC-IDS2018 (introduced)	Dataset construction and baseline ML evaluation	80+ features; 7 attack scenarios	No graph modelling; tabular ML only	Primary dataset source; validates benchmark choice
Wu et al.	2019	KDD Cup 99, NSL-KDD	Simplified GCN (SGC) for low-latency IDS	96% accuracy; low inference latency	Outdated datasets; no edge features	GCN baseline informs PAE-GNN benchmarking
Zhou et al.	2020	Multiple (survey)	Survey of GNN methods and applications	GNNs excel at relational reasoning tasks	General survey; not IDS-specific	Theoretical foundation for GNN applicability

Author(s)	Year	Dataset	Methodology	Key Results	Limitations	Relevance to PAE-GNN
						to traffic graphs

2.5. Review of Existing Systems

This section summarises the major operational network intrusion detection systems (NIDS) deployed within organisations. Understanding the limitations and strengths of current systems is important in positioning the contribution of PAE-GNN which is proposed to be an evolution of the architecture of graph-based detection rather than replacing the monitoring infrastructure currently employed by organisations. The review covers three widely adopted open-source NIDS systems and presents a review of the commercial Security Information and Event Management (SIEM) platforms which combine detection outputs from a variety of sources. The table 2.2 demonstrates the relative comparison of the existing systems with PAE-GNN in key detection capabilities.

2.5.1. Signature-Based NIDS: Snort and Suricata

Snort is currently the most widely used open source NIDS system; there are millions of active deployed instances of Snort in enterprise and government environments worldwide (Roesch, 1999). Snort is essentially a signature based detection system that uses a rule language that security managers use to match payloads, headers and flow attributes of packets to known attack signatures. The Snort rule set is actively maintained by Cisco Talos and the open-source community providing the most comprehensive source of signatures for thousands of attacks signatures (Cisco Talos). The single biggest weakness of Snort is that the detection is based upon attack signatures; if the attack signature is not present in the rule set, it will not trigger an alert and the false negative for a new attack signature is likely limited by the size of the rule set. Updating rulesets is another factor which would require updates to be deployed periodically; there may be some time before a new attack mutation has an attack signature that would be covered by an update to a Snort rule set.

Suricata, developed by the Open Information Security Foundation, enhances the Snort detection model by incorporating native multi-threading support, which facilitates high-throughput detection at speeds exceeding ten gigabits per second. Furthermore, it includes

application-layer protocol analysis for over fifty protocols, such as HTTP, TLS, DNS, and SMB (OISF, 2023). Suricata supports the Snort rule syntax while adding application-layer matching capabilities, enabling rule evaluation against decoded protocol fields rather than solely against raw packet bytes. Machine learning integration has been progressively introduced through community plugins, but the core detection engine of Suricata remains signature-dependent. Like Snort, Suricata does not incorporate graph-structured modelling of host communication patterns; the system evaluates each packet or flow in isolation, rendering it structurally unable to detect multi-host attack topologies that manifest only when multiple flows are considered jointly (Ferrag et al., 2020).

2.5.2. Behavioural Analysis Platform: Zeek

Zeek was first developed at the International Computer Science Institute and is unique among the NIDS platforms in that its main role is as a network analysis and logging platform rather than an alerting system (Paxson, 1999). Zeek has a scripting language that is well suited for protocol analysis and is probably what is responsible for the creation of organized logs detailing network activity. These logs are printed in JSON or tab-separated format and record DNS queries, HTTP requests, TLS handshakes, connection summaries, and many other protocol events. These structured logs serve as rich input data for downstream analytics platforms including SIEM systems and machine learning pipelines, and the CICFlowMeter features used in the CSE-CIC-IDS2018 dataset bear functional similarity to the flow records that Zeek generates in operational deployments.

Zeek's contribution to the IDS landscape is its structured, semantically rich logging rather than real-time detection; the platform does not natively raise intrusion alerts but provides the data infrastructure on which detection models operate. This positions Zeek as a natural data source for GNN-based detection pipelines: Zeek flow logs can be consumed by a graph construction module to produce the attributed directed graphs that the PAE-GNN processes. The subsequent integration of the PAE-GNN with logs produced by Zeek might offer a real-world deployment option that would not require changes to the current monitoring setup. This setup would only need to add a graph construction and inference layer just after the Zeek logging pipeline.

2.5.3. Commercial SIEM Platforms

Commercial Security Information and Event Management (SIEM) platforms, including Splunk, IBM QRadar, and Microsoft Sentinel, aggregate data from multiple sources

including NIDS platforms, endpoint detection systems, authentication logs, and cloud service telemetry to provide unified security monitoring dashboards with correlation-rule-based alerting and machine learning-assisted anomaly detection (Thakkar & Lohiya, 2021). These platforms offer powerful aggregation, visualisation, and compliance reporting capabilities but rely on rule-based correlation logic and statistical anomaly detection rather than graph neural network-based traffic modelling. The integration of GNN-based detection modules within SIEM data pipelines represents an active area of research and a natural deployment pathway for architectures such as the PAE-GNN (Bilot et al., 2023).

The web-based monitoring dashboard designed in this work is conceptually similar to SIEM user interfaces in that it provides a visualization interface for the outputs of an anomaly detector, allowing security analysts to interact with the model pipeline without directly interacting with the model pipeline. The key difference is the graph-based visualization user interface that is likely included by default for the PAE-GNN dashboard, which might show a graph visualization of communicating hosts with discrete attack labels for each edge - a relational context that a list of SIEM alerts does not offer. This design choice indicates a preference for making the graph-structured representation of network behaviour interpretable to analysts.

Table 2.2: Comparison of Existing NIDS Systems and PAE-GNN

Feature	Snort	Suricata	Zeek	GNN-Based IDS (PAE-GNN)
Detection Paradigm	Signature-based	Signature + App-layer	Behaviour logging	Graph-structured anomaly/classification
Zero-Day Detection	No	Limited	Indirect (via logs)	Yes — learned from flow patterns
Graph Topology Awareness	No	No	No	Yes — core architectural feature
Port-Aware Detection	Rule-defined only	Rule-defined only	Logged; not modelled	Yes — embedded as learned representation
Multi-Class Output	Alert/No Alert	Alert/No Alert	Log category	Yes — edge-level class prediction

Feature	Snort	Suricata	Zeek	GNN-Based IDS (PAE-GNN)
Deployment Interface	CLI / SIEM integration	CLI / SIEM integration	Log files / SIEM	Web dashboard (FastAPI + React)
Update Mechanism	Signature rule updates	Signature rule updates	Script updates	Model retraining on new labelled data

2.6. Gaps Identified in Literature

The analysis of relevant concepts, related works, and existing systems given in Sections 2.2-2.5 indication four significant gaps in the literature that collectively constitutes the research area that PAE-GNN aimed to solve and these are outlined and described as the explicit research justification for architectural and methodological decisions made in Chapters Three and Four.

The first and most significant gap is the absence of explicit destination port embeddings in existing GNN-based intrusion detection architectures. As documented across the related works reviewed in Section 2.4, studies including Lo et al. (2022), Li et al. (2023), and Pujol-Perich et al. (2022) incorporate edge features to varying degrees but do not treat destination port as a semantically privileged feature warranting a dedicated learned embedding. Where port information is included in feature sets, it is typically encoded as a raw integer or one-hot encoding among other continuous flow statistics, without any mechanism to learn the service-semantic relationships between ports or to generalise across port-specific attack patterns. This omission is consequential: as established in the theoretical framework in Section 2.3, destination port is among the most diagnostically informative features in network flow classification, directly identifying the targeted service and providing the primary discriminating signal between attack classes that target different protocols. The PAE-GNN addresses this gap through a dedicated port embedding layer that maps destination port integers into a continuous learned representation space incorporated within the edge-conditioned message-passing process.

The second gap is the predominance of binary classification formulations in the GNN-based IDS literature. As identified by Bilot et al. (2023) and confirmed by the individual study

reviews in Section 2.4, the overwhelming majority of GNN-based IDS architectures evaluate only binary benign-versus-malicious classification, with multi-class discrimination between specific attack categories receiving substantially less attention. The gap is not in coverage of a benchmark but merely in binary classification not meeting the operational need for the SOC. The specific counter-measure to be deployed is heavily dependent on the actual category of attack found. For example, DDoS sets would require rate limiting and upstream scrubbing while brute-force authentication campaigns would likely need account lockouts, and multi-factor authentication enforcement. In similar fashion, web application attacks would need the web application firewall rules updated and the web application itself patched. A binary malicious flag simply cannot support these differentiated responses and a multi-class formulation of the task is required, which is precisely what PAE-GNN is attempting with it's multi-class attack categories classification for CSE-CIC-IDS2018.

The third gap identified is the limited number of deployment-focused, end-to-end GNN-based IDSs that encapsulate trained models into easy-to-use operational monitoring dashboards. The majority of the existing literature that we reviewed in 2.4 embodies experiment classification results in a research setting without showing that trained models have been embedded into tools open to security practitioners without machine learning expertise. We hypothesize that this disconnect between the research contributions and their deployment feasibility will limit the practical impact of the algorithmic improvements. Security operations staff may not be able to use graph-learning-based detection methods if they are only experimental scripts that require specialized skill to run. In this study, we attempt to remedy this gap by designing an end-to-end deployment pipeline, that incorporates PAE-GNN into a browser-based web UI for monitoring, exemplifying a reference architecture for practical GNN-based IDS.

The fourth gap is the limited empirical coverage of the CSE-CIC-IDS2018 dataset in the GNN-specific literature. While CICIDS2017 has been evaluated by numerous GNN-based studies including Lo et al. (2022), Caville et al. (2022), and Tang et al. (2022), the more recent CSE-CIC-IDS2018 dataset, which captures a broader range of contemporary attack scenarios, has received comparatively limited attention from the graph learning community despite its wider use in tabular ML-based IDS research (Sharafaldin et al., 2018). The PAE-GNN evaluation on CSE-CIC-IDS2018 contributes to filling this empirical gap by documenting GNN classification results on a benchmark that is increasingly adopted as the

reference dataset for network intrusion detection research but remains undercharacterised in the graph-structured learning literature.

2.7. Summary of Literature Review

This review has covered the theoretical and empirical body of work relevant to the PAE-GNN in four key areas: network security and IDS, machine learning in IDS, graph theory and GNNs, and the relevant work applying GNNs in IDS. While it is apparent that the field has made huge progresses from signature-based detection, through tabular ML to graph-structured ML, it is also certain that specific, meaningful research gaps will remain, such as the lack of destination port embeddings in the existing GNN-based IDSs, the strong focus on binary classification formulations, the underused implementational and deployment-ready methodologies, and the relative dearth of empirical coverage of the CSE-CIC-IDS2018 in the literature of applying GNNs in IDS. This research gap is where the PAE-GNN sits (Bilot et al., 2023; Lo et al., 2022).The theoretical/ conceptual framework we have provided in Section 2.3 explains the conceptual and theoretical motivation behind the three conceptual foundations of the PAE-GNN: the domain principle of port-aware service semantics, explains why destination port was chosen as the principle embedding feature; the network principle of graph theoretic network modelling, explains why network traffic was modelled as an attributed directed graph; and the learning principle of message-passing neural computation, explains why a message-passing approach to learnt graph representation was adopted. The domain, conceptual and theoretical commitments, have been translated into the construction schema, edge-conditioned architecture and port embedding scheme in a principled way in the build schema, as described in Chapter Three (Gilmer et al., 2017).

Further, the systems survey established that operational NIDS platforms, Snort, Suricata and Zeek, fill the monitoring role but cannot, due to their fundamental architecture, conduct GNN architecture-based, graph-topology-level detection. Commercial SIEM platforms, on the other hand, offer aggregation and correlation suitable for multi-product ecosystem integration, which could incorporate GNN-based detection modules. Gaps identified in research combined, under the following requirements description, to support a diagnosis for the design strategy of PAE-GNN and in the evaluation work of chapters 3 and 4, in particular, port-aware, edge-centric, multi-class GNN-based IDS, deployment-oriented implementation.

CHAPTER THREE
RESEARCH METHODOLOGY

3.1. Introduction

This chapter explains the adopted research methodology that forms the basis of the studied Port-Aware Edge-Feature Enhanced Graph Neural Network (PAE-GNN) for network traffic anomaly detection. This research is classified as a mixed research, for it combines the character of applied system development research with embedded evaluative research. In more specific terms, the process involves designing and building an operational detection system, followed by investigating that in a set of controlled experiments, where a baseline and a systematic ablation study are performed. This combined nature of research is introduced because of the two basic objectives of the project: the first aims to produce an operational intrusion detection system, while the second aims to generate empirical evidence about the port-aware edge feature's role in graph neural networks' classification performance.

The methods follow the order of the six research goals described in Section 1.3, starting with collection and pre-processing of data sources, construction of graph representations, design of graph classification architectures, evaluation design, and deployment as a dashboard. Within each section, the design choices made, technical steps followed, and implementation choices adopted are detailed. This pipeline was implemented as a machine learning pipeline of three stages, one each for pre-processing, model training, and ablation testing, with a web based deployment layer. Training was performed on a Google Colab GPU with the CSE-CIC-IDS2018 dataset, while development and inference testing was carried out on a Windows local machine.

The preprocessing pipeline converts raw network flow records (CSE-CIC-IDS2018) into attributed directed graphs suitable for edge-level classification with GNN. The model training stage trains a PAE-GNN with edge-conditioned message passing and port-aware feature embeddings, in addition to a node-only GraphSAGE baseline, to facilitate benchmarking. An ablation study systematically varying the edge feature configuration allows quantification of the impact of port-aware features on classification accuracy, following the work of Lo et al. (2022). A deployment stage has a FastAPI RESTful backend exposing the PAE-GNN model for inference, and a React single-page application for forensic analysis and live traffic visualization.

3.2. Research Design and System Requirements

The research adopts a mixed-method approach combining an applied design with an evaluative investigation. The applied design component covers the design of a novel Graph Neural Network (GNN) architecture, called the PAE-GNN, along with its integration into a detection and monitoring pipeline. The evaluative part involves the quantitative measurement of the classification performance of the PAE-GNN relative to that of a baseline model and to an artificially controlled feature ablation scenario. The combination of design and evaluation methods is considered appropriate because the research questions can only be adequately answered by both designing a working artefact for producing experimental evidence and by designing a controlled empirical evaluation (Thakkar & Lohiya, 2021).

We used the CSE-CIC-IDS2018 benchmark dataset for this work. Data was selected for the presence of labeld realistic multi-attack network flow records, suitable for supervised multi-class intrusion detection. Unit of analysis was the directed edges of the host communication graph, i.e. individual network flows. The dependent variable was the edge-level class label assigned by the model, while the independent variable manipulated in the ablation study was the composition of the edge feature vector, namely port-aware vs. port-unaware features. All experiments used a fixed stratified train/validation/test split (seed 42), such that experiments with differnet model configurations are directly comparable.

Internal validity is preserved by conducting each experiment under the same conditions; the PAE-GNN, the NodeOnlyGNN baseline, and each ablation experiment are trained and tested using the same dataset, stratified split, class weighting, and optimizer conditions. Consequently, it can be asserted that performance differences may be attributed solely to the independently varied variable-the edge feature vector composition. Test reliability is secured by using a fixed, pre-selected random seed (randomstate=42 and torch.manualseed(42)) during preprocessing, graph construction, data splitting, and model initialization such that observed results can be repeated with the same code base and conditions. The main external validity threat involves the evaluation of the PAE-GNN on a single benchmark dataset; empirical validity of the generalization of the performance results to other network environments, traffic distributions, and attack types not present within the CSE-CIC-

IDS2018 dataset is therefore not demonstrated and is one of the limitations discussed in Chapter Five.

3.2.1. Functional Requirements

The functional requirements of the PAE-GNN define the way data processing, model training, evaluation and dashboard functionalities the entire system should perform. In total nine functional requirements were defined of this system and the full pipeline from dataset preprocessing to real-time visualization of threats on the deployed dashboard. The functional requirements are based on the six study objectives and a functional analysis of the environment of a deployed network intrusion detection system following Thakkar and Lohiya (2021). In addition, all functional requirements are detailed in Table 3.1.

Table 3.1: Functional Requirements

FR	Requirement	Description	Priority
FR1	Dataset Preprocessing	The system shall load all CSE-CIC-IDS2018 CSV files, construct pseudo-IP node identities where real IP columns are absent, extract fourteen edge features, and apply SMOTE balancing	High
FR2	Graph Construction	The system shall construct a directed attributed graph in which unique IP addresses form nodes and individual network flows form directed edges enriched with the fourteen-dimensional feature vector	High
FR3	Model Training	The system shall train the PAE-GNN model with edge-conditioned message passing, port-aware embeddings, and	High

FR	Requirement	Description	Priority
		balanced cross-entropy loss, saving the best checkpoint based on validation macro-F1	
FR4	Edge-Level Classification	The system shall classify each network flow edge as Benign or as a specific attack class, producing a predicted class label and an attack probability score for each flow	High
FR5	Baseline Comparison	The system shall train and evaluate a node-only GraphSAGE baseline under identical conditions for direct performance comparison with the PAE-GNN	Medium
FR6	Ablation Study	The system shall evaluate six distinct edge feature configurations to quantify the individual contribution of port-aware features to classification performance	Medium
FR7	CSV Forensic Analysis	The dashboard shall accept uploaded CIC-format CSV files, run inference via the FastAPI backend, and render classified flows as an interactive graph	High
FR8	Live Network Monitoring	The dashboard shall support real-time packet capture via a configurable network interface, processing captured flows	High

FR	Requirement	Description	Priority
		through the PAE-GNN and updating the threat graph every 2.5 seconds	
FR9	Threat Visualisation	The dashboard shall render the host communication graph with colour-coded severity (benign, suspicious, malicious), node sizing by degree, and edge width proportional to attack confidence	High

The requirements are grouped to reflect the order of data flowing into the system. Requirements FR1 and FR2 are related to the data preparation and graph construction component that produces the attributed graph for model training. Requirements FR3 and FR4 specify the critical model training and inference component which constitutes the most probable contribution of the research. Requirements FR5 and FR6 specify the comparison and ablation evaluation criteria which might empirically verify the effectiveness of the PAE-GNN design choices. Meanwhile, requirements FR7 to FR9 specify the dashboard requirements that indicate the deployment oriented contribution of this study.

3.2.2. Non-Functional Requirements

These non-functional requirements are essential in controlling the performance, scalability, reproducibility, usability, and security characteristics of the PAE-GNN system. They probably have control over implementation decisions made at each stage of the pipeline and dashboard. These are mainly related to the reproducibility of the experiment and accessibility of the deployed dashboard by laymen. Besides these, table 3.2 shows the seven non-functional requirements identified for the system.

Table 3.2: Non-Functional Requirements

NFR	Category	Requirement	Priority
NFR1	Performance	The model shall complete training within 100 epochs on Google Colab GPU	High

NFR	Category	Requirement	Priority
		hardware, with early stopping triggering before epoch 100 if validation macro-F1 plateaus for 25 consecutive epochs	
NFR2	Scalability	The preprocessing pipeline shall support loading up to 10 CSE-CIC-IDS2018 CSV files in chunks of 100,000 rows without memory overflow on standard Colab RAM allocation	High
NFR3	Reproducibility	All random operations shall use fixed seeds (random_state=42, torch.manual_seed(42)) to ensure reproducible preprocessing, graph construction, and model initialisation	High
NFR4	Usability	The dashboard shall be accessible via a standard web browser without requiring end-user installation of Python or machine learning libraries; all model inference shall be handled server-side	High
NFR5	Responsiveness	The live monitoring mode shall refresh the threat graph at intervals not exceeding 2.5 seconds, and the dashboard layout shall adapt to screen widths down to 720 pixels	Medium
NFR6	Maintainability	The system shall be structured as a three-stage pipeline (preprocessing, training, ablation) with separate scripts and clearly documented interfaces to facilitate independent modification of each stage	Medium

NFR	Category	Requirement	Priority
NFR7	Security	The FastAPI backend shall implement CORS middleware restricting cross-origin requests to permitted localhost origins, and all file uploads shall be validated for CSV format before processing	Medium

Among the non-functional requirements, NFR3 reproducibility through fixed random seeds is particularly significant for a research implementation, as it ensures that the experimental results reported in Chapter Four can be reproduced by other researchers using the same dataset and pipeline. NFR4 — browser-accessible inference directly addresses the deployment gap identified as a research gap in Section 2.6 by ensuring that the trained PAE-GNN is accessible without requiring end-user installation of the machine learning stack.

3.2.3. System Requirements

Table 3.3 and Table 3.4 document the hardware and software requirements for the PAE-GNN system across its two operational contexts: the model training environment and the dashboard deployment environment. The training environment requires GPU hardware for practical training times, whereas the dashboard deployment environment operates on standard CPU hardware given that inference on individual CSV batches does not require GPU acceleration at the flow counts typical of dashboard use.

Table 3.3: Hardware Requirements

Component	Specification	Purpose
Training Platform	Google Colab (NVIDIA T4 / A100 GPU, 12-40 GB VRAM)	Model training and ablation study execution
Development Machine	Windows 10/11, Intel/AMD CPU, 8 GB RAM minimum	Code development, dashboard testing, inference
Runtime Memory	Minimum 8 GB RAM for preprocessing; 12 GB recommended for full dataset loading	In-memory graph construction and SMOTE execution
Storage	Minimum 5 GB for CSE-CIC-IDS2018 CSV files and model checkpoints	Dataset storage and model checkpoint persistence

Table 3.4: Software Requirements

Library / Tool	Version	Role in System
Python	3.10+	Core programming language for all ML pipeline scripts
PyTorch	2.0+	Neural network implementation, tensor operations, GPU acceleration
PyTorch Geometric	2.3+	Graph data structures, MessagePassing base class, SAGEConv baseline
scikit-learn	1.3+	LabelEncoder, train_test_split, classification_report, F1 metrics
imbalanced-learn	0.11+	SMOTE oversampling, RandomUnderSampler pipeline
Pandas / NumPy	2.0+ / 1.24+	Data loading, feature extraction, numerical transformations
FastAPI	0.110+	RESTful backend API, file upload handling, live monitor endpoints
React	18+	Single-page frontend application, state management, UI rendering
Cytoscape.js	3.28+	Interactive graph canvas for host communication visualisation
Tailwind CSS	3.4+	Utility-first CSS framework for dashboard layout and styling
Lucide React	0.383+	Icon library for dashboard UI elements
Matplotlib / Seaborn	3.7+ / 0.12+	Training curve plots, confusion matrix heatmaps, ablation charts

The software requirements hint at the modular nature of the system, since: the machine learning pipeline dependencies (PyTorch, PyTorch Geometric, scikit-learn, and imbalanced-learn) are only required during training, and aren't installed on the client end user; the FastAPI backend requires Python and a selection of inference only libraries; and the React frontend is transpiled to static assets served separately from Python. Thus, it's likely that the dashboard could be deployed as a lean web app without including the full machine learning stack on production servers.

3.3. Data Collection and Pre-Processing

Data collection and preprocessing transforms raw CSE-CIC-IDS2018 CSV flow data into a class-balanced, normalized, graph-ready form of what is required for the PAE-GNN training pipeline (directly relating to Objectives i and ii of Sec. 1.3). It is carried out by step1cic2018smote.py. The data prepossessing pipeline consists of five successive sub-stages: loading of the dataset with pseudo-IP generation, extraction of the feature columns,

cleaning of data and merging of the target labels, feature engineering of the edge attributes, class balancing via SMOTE (Sharafaldin et al., 2018).

3.3.1. Dataset Acquisition

The CSE-CIC-IDS2018 dataset was obtained from its public distribution through the Canadian Institute for Cybersecurity. The publicly distributed version of the dataset provides ML-ready CSV files in which the raw packet captures have been processed by the CICFlowMeter tool to extract over eighty flow-level statistical features per connection record. A known characteristic of this public distribution is that nine of the ten available CSV files have had the source and destination IP address columns stripped, retaining only the derived flow statistics (Sharafaldin et al., 2018). Only one file in the distribution retains the original IP address columns. This distribution artefact necessitated a pseudo-IP construction strategy for the nine affected files.

The pseudo-IP construction scheme establishes a mapping from each network flow to a source destination node pair by exploiting the available port and protocol information. The scheme is academically justifiable because of the port-awareness design of PAE-GNN (Port-Aware Embedding Graph Neural Network). The scheme uses the following definition for source nodes and destination nodes respectively: 'client[protocol][srcportbucket]' and 'service[protocol][dstportbucket]'. Ports below 1024 are well-known and preserved as separate nodes; registered ports (between 1024 and 49151) are assigned into buckets of 100, resulting in significantly fewer nodes; ephemeral ports (above 49152) are bucketed in 1 for each protocol. This bucketing is important as the semantic meaning of service endpoints can be captured by shared nodes of the communication graph, which is port-aware of the PAE-GNN architecture. The one and only file with actual IP addresses was kept unmodified.

Additionally, all ten CSVs were read with a batch size of 100,000 rows in order to work around the training RAM constraints with Google colab. A fractional sampling approach was applied to files that had over 8 million rows in order to not overtake the RAM buffer; each chunk was sampled at a rate of min(1.0, sampleperfile / 8,000,000, with a default value for sampleperfile of 500,000. The sampling method here should keep the class imbalance the same within the sample and also fit in the available RAM for the colab runtime (Ferrag et al., 2020).

3.3.2. Data Cleaning and Label Consolidation

The raw flow records were loaded and then cleaned to account for missing values, and inf values, using label mapping. Numeric columns were cast to the int type, and values that could not be parsed were replaced with NaN. Rows where any of the numeric feature columns contained NaN or inf were then removed. This should be fairly effective at removing corrupted flow records that are created when CICFlow Meter encounters a zero-duration flow, or is forced to divide by zero when calculating rate features, known artefacts of the CIC-IDS2018 generation process (Sharafaldin et al., 2018).

The raw label column from the CSE-CIC-IDS2018 dataset included attack subtype labels such as DDoS attacks-LOIC-HTTP, DoS attacks-Hulk, and FTP-Brute Force. These were combined into five attack meta-categories: Benign, DDoS, DoS, Brute Force, and Bot. Attack subtype labels that did not fit into any of these meta-categories such as Infiltration attacks and SQL Injection were given the label Other Attack and subsequently removed from the dataset. SQL Injection was not included because the number of samples in the publicly available subset was too low to apply SMOTE with k=5 nearest neighbours (fewer than six samples per class cannot reliably employ nearest-neighbour oversampling). Any other categories with less than the minimum number of samples were similarly removed.

3.3.3. Edge Feature Engineering

Fourteen edge features were extracted from each cleaned flow record to form the edge attribute vector associated with each directed graph edge. The feature set was designed to capture three distinct types of information relevant to network intrusion classification: port-aware service semantics, network-layer protocol identity, and flow-level statistical behaviour. The first five features constitute the port-aware feature group that represents the primary novel contribution of the PAE-GNN; the remaining nine features constitute the flow statistics group that provides the behavioural context in which port-aware information is interpreted. Table 3.5 documents all fourteen features with their source columns, transformations, and semantic roles.

Table 3.5: PAE-GNN Edge Feature Vector Specification

#	Feature Name	Source Column	Transformation	Semantic Role
1	Normalised Source Port	Src Port	value / 65535	Port-aware: originating endpoint identity
2	Normalised Destination Port	Dst Port	value / 65535	Port-aware: targeted service identity (key feature)
3	Well-Known Port Indicator	Dst Port	Binary: 1 if port ∈ {20,21,22,23,25,53,80,110,143,443,8080}	Port-aware: flags standard service-port targeting
4	Ephemeral Port Indicator	Dst Port	Binary: 1 if dst_port > 49152	Port-aware: distinguishes ephemeral from service ports
5	Normalised Port Distance	Src Port, Dst Port	|src − dst| / 65535	Port-aware: relative port asymmetry signal
6	Normalised Protocol	Protocol	value / 17	Network layer: transport

#	Feature Name	Source Column	Transformation	Semantic Role
				protocol identifier
7	Forward Packet Count	Tot Fwd Pkts	log1p(value)	Flow stat: transmission volume forward direction
8	Backward Packet Count	Tot Bwd Pkts	log1p(value)	Flow stat: response volume backward direction
9	Forward Byte Volume	TotLen Fwd Pkts	log1p(value)	Flow stat: total bytes transmitted forward
10	Backward Byte Volume	TotLen Bwd Pkts	log1p(value)	Flow stat: total bytes received backward
11	Flow Duration	Flow Duration	log1p(value)	Flow stat: connection lifetime in microseconds
12	Bytes Per Second	Flow Byts/s	log1p(value)	Flow stat: throughput rate indicator

#	Feature Name	Source Column	Transformation	Semantic Role
13	Packets Per Second	Flow Pkts/s	log1p(value)	Flow stat: packet transmission rate
14	Mean Forward Packet Length	Fwd Pkt Len Mean	Raw value	Flow stat: average payload size forward direction

Log1p transformation computing $\log(1 + x)$ was applied to all count-based and rate-based flow statistics to compress the heavy-tailed distributions characteristic of network traffic data, where the majority of flows have small values but a minority generate extremely large packet counts or byte rates during flooding attacks (Lo et al., 2022). The destination port was also normalized between 0 and 1 by dividing over the maximum port number 65535, so that can be directly compared in magnitude to other normalized features. The destination port, well-known port indicator and ephemeral port indicator encode knowledge about the semantics of ports so that the model does not need to learn such knowledge by itself.

3.3.4. Class Balancing with SMOTE

Class imbalance is a problem for the CSE-CIC-IDS2018 dataset, as some of the attack classes may only be a small percentage of the total number of packets in the dataset. When trained directly on the imbalanced distribution, a classifier optimises for the accuracy of the majority class and exhibits the worst case of failure: it does not learn discriminative representations of attack classes in the minority. This is the failure mode that is most critical for an operational intrusion detection (Thakkar & Lohiya, 2021). To the training set feature matrix, synthetic minority class samples were created using Synthetic Minority Over-sampling Technique (SMOTE) which involves interpolating between pairs of existing minority class samples in the 14-dimensional feature space.

The SMOTE configuration was set as k_neighbors = min(5, min_class_size − 1) which parameterizes the neighbourhood size based on the number of available training instances per class, with a lower number if the class size was small. The oversampling target was set to be the minimum of MAX_SAMPLES_PER_CLASS and max(class_counts.values()) (defaulting to 50,000), which makes sure that no class goes past the maximum allowed number of samples, and that oversampling pulls minority classes up to the size of the largest class, not to a fixed number. SMOTE was followed by a RandomUnderSampler to reduce all classes to a fixed size, resulting in a balanced dataset with equal number of samples for each class retained. Secondly, during training, a balanced class weight formula was also used to compute the class weights for the CrossEntropyLoss function, which are computed as n_samples / (n_classes × class_count) from scikit-learn, thus introducing a second layer of class balance correction at the loss computation level (Ferrag et al., 2020).

3.3.5. Graph Construction and Train/Validation/Test Split

The balanced, feature-extracted flow data sets were dumped to a PyTorch Geometric Data object, which encodes the host communication graph. All unique source and destination IP address strings that appear in any of the flow records are picked out and assigned as integer node indices, which form the node set \(V\) of the directed graph. Each flow record is then represented as a directed edge between the source IP node index and destination IP node index, which form the edge set \(E\) of the directed graph. Thus, the node-to-edge tensor flow consists of two columns: the shape of edgeindex is [2, numedges],and each column encodes a directed source-to-destination relation. While the flow has 14 features, the shape of the edgeattr is [numedges, 14].

Node features are extracted solely based on graph structure and not on external attributes, because the CSE-CIC-IDS2018 flow records lack any host-level information beyond the ip addresses to identify the nodes. Each node is assigned a four-dimensional feature consisting of the log1p out-degree, in-degree, and total degree, with a fourth all-zero padding dimension. The degree-based node features probably provide some information to the message-passing process related to the activity level of each host in the graph, which might allow the GNN to recognize network hubs and leaf nodes based solely on traffic volume. Lastly, the data is split into training, validation, and test sets in a 70/15/15 ratio with stratified

sampling to maintain class distributions across all three splits and a seed of 42 for full reproducibility.

3.4. System Architecture

We will start with an explanation of the PAE-GNN architecture and our baseline model, which is the NodeOnlyGNN (Section 2.2). Next follows the explanation of the training process used for the baseline, and PAE-GNN models. NodeOnlyGNN is our baseline model, used for comparison of performance. The general GNN architectures presented in the first two articles in Section 2.2 were node-based models, where node features are used as input to the model. Section 2.6 highlights the reasoning behind why the proposed model is a GNN, incorporating port-aware edge embeddings, and what were identified as the shortcomings of the existing GNN-based IDS literature (Bilot et al., 2023).

Figure 3.1: PAE-GNN Complete System Architecture

3.4.1. EdgeAwareGNNLayer

The core computational unit of the PAE-GNN is the EdgeAwareGNNLayer, a custom message-passing layer implemented by extending PyTorch Geometric's MessagePassing base class with mean aggregation. The layer accepts three inputs at each forward pass: a node feature matrix X of shape [num_nodes, node_in_channels], an edge connectivity tensor edge_index of shape [2, num_edges], and an edge attribute matrix edge_attr of shape [num_edges, edge_in_channels]. The layer implements the message-passing framework of Gilmer et al. (2017) through three sequential operations.

The first operation applies the node Multi-Layer Perceptron (MLP) to the full node feature matrix, transforming each node's input representation through a linear layer, ReLU activation, Dropout (p=0.2), and a second linear layer, producing transformed node embeddings of dimension out_channels. The second operation computes per-edge messages by passing each edge's feature vector through the edge MLP — with the same two-layer architecture — and concatenating the resulting edge embedding with the transformed source node embedding, then projecting the concatenated (2 x out_channels)-dimensional vector down to out_channels via the combine MLP. This concatenation is the mechanism through which edge attributes condition the message propagated from each source node to its neighbour, implementing the edge-conditioned message function of the MPNN framework.

The third operation aggregates all incoming messages at each destination node using mean pooling, producing updated node embeddings of dimension out_channels.

3.4.2. PAE-GNN Architecture

The full PAE-GNN model stacks two EdgeAwareGNNLayer instances followed by an edge classification head. The first convolutional layer (conv1) maps from node_features (4) to hidden_dim (128) with edge features of dimension 14, producing intermediate node embeddings of size 128. A ReLU activation is applied to the output of conv1 before it is passed to the second convolutional layer (conv2), which maps from hidden_dim (128) to hidden_dim (128), again conditioning on the full fourteen-dimensional edge feature vector. Two message-passing layers were chosen to enable each node's representation to encode information from its two-hop neighbourhood, capturing the relay-host topology characteristic of certain DDoS and botnet communication patterns without the over-smoothing that deeper networks introduce on dense graphs (Zhou et al., 2020).

The edge classification head receives a concatenated representation for each edge comprising three components: the source node embedding x[row] of dimension 128, the destination node embedding x[col] of dimension 128, and the original edge feature vector edge_attr of dimension 14, forming a 270-dimensional input vector per edge. This concatenation ensures that the port-aware edge features are directly available to the classifier at prediction time, not merely used during message propagation, providing a residual connection between the raw port features and the final classification decision. The classifier applies a three-layer MLP: a linear layer (270 to 128) with ReLU and Dropout (p=0.3), a second linear layer (128 to 64) with ReLU and Dropout (p=0.3), and a final linear layer (64 to num_classes) producing per-edge classification logits. The PAE-GNN architecture is illustrated in Figure 3.2.

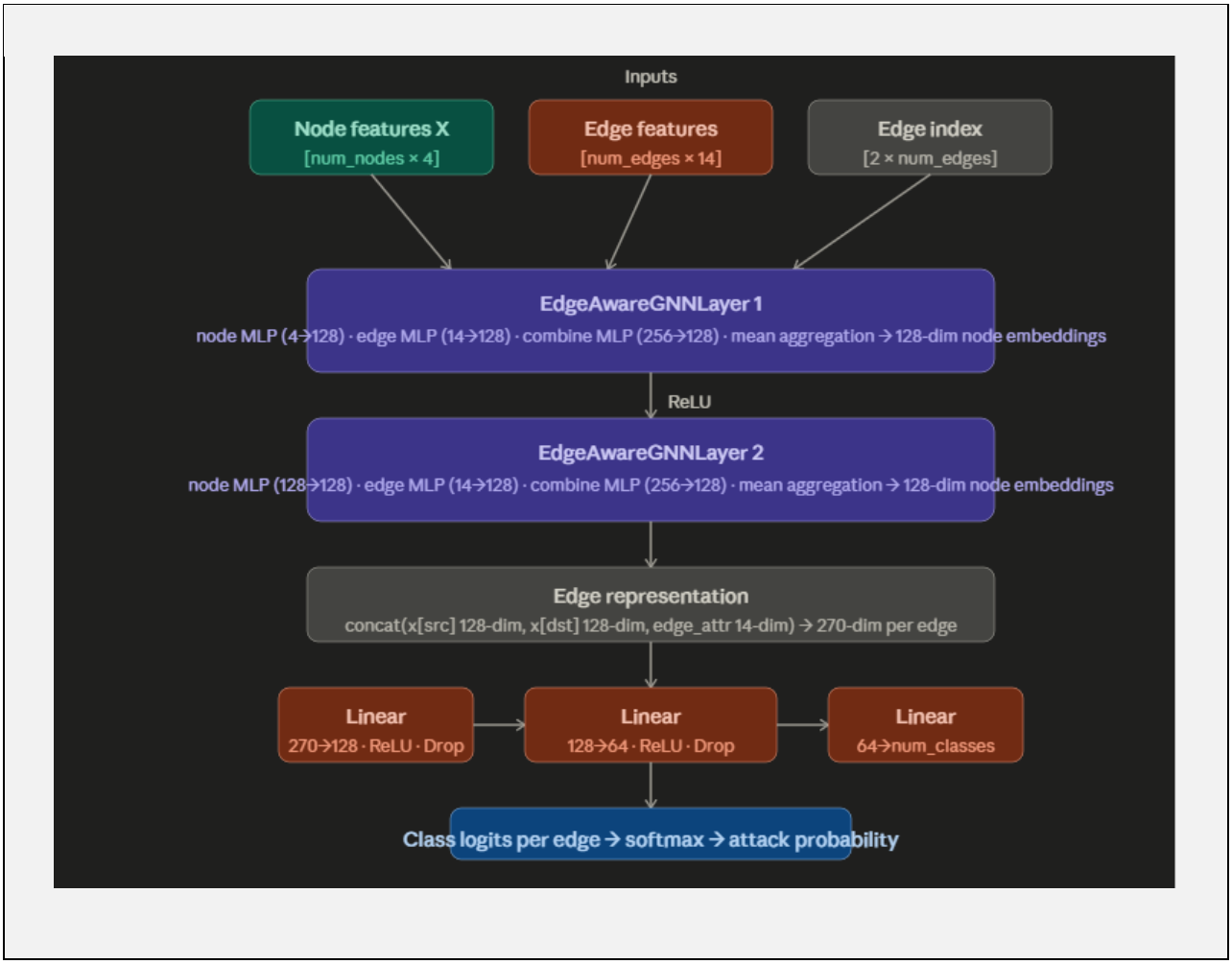


Figure 3.2: PAE-GNN Architecture — Two EdgeAwareGNNLayers followed by Port-Aware Edge Classifier

3.4.3. Node-Only Baseline Architecture

The NodeOnlyGNN baseline is a two-layer GraphSAGE model implemented using PyTorch Geometric's SAGEConv operator. The first SAGEConv layer maps from node_features (4) to hidden_dim (128) and the second maps from 128 to 128, with ReLU activations applied after each layer. The edge classification head receives only the concatenated source and destination node embeddings a 256-dimensional vector with no access to edge features, implemented as a two-layer MLP (256 to 128 to num_classes) with ReLU and Dropout (p=0.3). The goal of this baseline is to use the identical hidden dimensionality, classification head depth, and training scheme as in PAE-GNN so that the performance difference should mostly come from the edge-aware, port-informed message-passing paradigm instead of the model capacity or optimization setting.

3.4.4. Training Procedure

Both models were trained using the Adam optimiser with a learning rate of 3x10^-4 and weight decay of 5x10^-4. The loss function is weighted Cross-Entropy Loss, where per-class weights are computed using the scikit-learn balanced formula: weight_c = n_samples / (n_classes x class_count_c). This formula assigns higher loss weight to minority classes, encouraging the model to learn discriminative boundaries for attack classes that remain underrepresented even after SMOTE balancing. Training proceeded for a maximum of 100 epochs with early stopping triggered when the validation macro-F1 score failed to improve for 25 consecutive epochs, after which the model checkpoint achieving the best validation macro-F1 was restored for evaluation on the held-out test set. All training was performed using Google Colab GPU hardware, with a NVIDIA T4 or A100 available when required. The research employed a full-batch training approach for all graph data loaded onto the GPU. The use of full-batch training rather than mini-batch training was motivated by the size of the entire balanced graph - 4,964 nodes and 250,000 edges - easily stored in the available GPU memory. Additionally, the loading of the whole graph in a single batch implies various benefits: no neighbourhood sampling is necessary, the amount of gradient noise introduced by the mini-batch approximation is reduced, and every message passing step takes into consideration the whole graph topology. All three reasons make sense to us for an edge-classification task, where the context of each flow is informative.

3.5. Ablation Study Design

To address Objective v, the ablation study aimed to prove evidence for the individual impact of the port-aware features on the classification performance of PAE-GNN. The edge vector has been divided into fourteen different features and six different feature configurations were defined by systematically removing or isolating feature subsets from the full vector. A new PAE-GNN model was instantiated for each configuration with the input edge dimension, and trained from scratch for 100 epochs, with identical hyperparameters (learning rate (lr) is 3 x 10^-4, and patience is 25), and then tested on the same held out test set. This is a controlled experimental design that will allow you to understand the difference between performance in different configurations and that it will not be due to training variability (Bilot et al., 2023).

All six ablation configurations are listed in Table 3.6. All features is the full system configuration (C1) that is used as a reference point to compare the other configurations. The

five port-associated features (indices 0-4) are removed from the configuration and only the protocol and flow statistics features are retained, which directly quantifies the total contribution from all port information (configuration C2, No Port Features). Port Features Only (C3) only implements the six port and protocol features, and assesses if a port-aware infrastructure with only port information is enough to provide a signal for attack classification. The performance of a model that is equivalent to a conventional tabular ML approach, without the port awareness, is obtained by keeping only the eight flow statistical features (indices 6-13) in Configuration C4, Flow Statistics Only. Configurations C5 and C6 achieve finer-grained ablation, the former to test the contribution from the binary well-known port indicator and the latter to evaluate the minimum viable port-aware configuration.

Table 3.6: Ablation Study Feature Configurations

Config	Name	Features (n)	Feature Indices	Research Question Addressed
C1	All Features (Full PAE-GNN)	14	0-13	Baseline full-system performance
C2	No Port Features	9	5-13	What is the total contribution of all port features?
C3	Port Features Only	6	0-5	Can port + protocol alone classify attacks?
C4	Flow Statistics Only	8	6-13	Baseline ML performance without port awareness
C5	No Well-Known Port Indicator	13	0-1, 3-13	Does binary well-known port flag add value over raw port?
C6	Minimal (Dst Port + Protocol)	2	1, 5	Minimum viable port-aware configuration

Primary comparisons of interest are between configurations C1 and C2, to measure the impact of port-aware features on overall accuracy and macro-F1, and secondary comparison of C1 and C4, to measure the combined impact of port features and protocol information relative to the flow statistics-only baseline. Per-class F1 across all 6 configurations is examined to determine which attack classes receive the most benefit from port-aware

features, to empirically verify that port information is likely to be most discriminative for attack classes that are specific to network services.

3.6. Web Application Design

The online monitoring dashboard leverages the trained PAE-GNN model into a deployable operational interface, which seems to fulfill Objective vi in Section 1.3 and seems to fulfill the deployment gap scenario in Section 2.6. The dashboard is a two-layer web application, where the REST API inference endpoints are exposed in a FastAPI back-end web server, and a React single-page application is used as the user interface. The deployment architecture cleanly isolates all model inference code in the server-side so that the end user is likely to only need a general browser to deploy and operate this system, without needing to install a local Python environment or machine learning tools.

3.6.1. Backend Architecture

The backend was implemented using the FastAPI framework as it ships with automatic OpenAPI documentation, asynchronous request handling and request validation provided by Pydantic. All inference operations with the model are carried out within the Nids Inference Engine class. This includes loading the trained PAE-GNN checkpoint, inferring architectural dimension of the model based on the contents of the checkpoint state dictionary, and getting class name labels based on the contents of the metadata checkpoint. The inference engine exposes the analyze_dataframe method that takes as input a Pandas Data Frame of flow records, constructs the attributed directed graph in-memory, forward propagates it through the loaded model, computes softmax to get class probabilities, and then returns a result containing classified edges, node statistics and aggregated metrics.

The provided backend exposes 7 REST endpoints divided into two groups of functionality. The health/csv analysis group consists of GET /api/health (yielding the model status and class names) and POST /api/analyze-csv (the file upload, with optional parameters of confidence threshold and max flow count). The other group, the live monitoring system, consists of GET /api/live/status, GET /api/live/interfaces, POST /api/live/start, POST /api/live/stop, and GET /api/live/events. The live monitoring system launches a Packet capture subprocess via the live_sniffer.py script that may output flows to a shared CSV file, then the Live Monitor class polls, with configurable frequency, ingests any newly received

rows with the Nids Inference Engine, and stores classified flow events in a thread-safe ring buffer, presumably limited to 120 events with a 5-minute TTL. Finally, the CORS middleware is set up to accept requests from localhost dev origins over ports 5173 and 5174.

3.6.2. Frontend Architecture

Application frontend is developed as a React single-page app using functional components and hooks. The frontend organizes its components in a three-panel layout: a left filter rail for configuration controls, a central graph canvas for interaction with threat maps, and a right detail panel for viewing the incoming node or flow selections. The app operates in two modes, selected by a segmented control: Forensic mode and Live Map mode. In Forensic mode, the user uploads a CIC-format CSV file, and the frontend forwards it to POST /api/analyze-csv and displays the classified flows in an interactive graph. In Live Map mode, the frontend launches live monitoring by POST /api/live/start and polls GET /api/live/events every 2.5 seconds for new classified flows, which are then displayed as a graph.

The graph visualization in the application is built on Cytoscape.js, a high performance graph visualization engine that enables rendering of an interactive force-directed graph with node dragging, scroll-to-zoom, and element selection. Nodes are scaled to their degree in the communication graph and colored to indicate severity – benign nodes in blue, suspicious nodes in amber, and malicious nodes in red, with the monitored host device scaled to white with a red border in live mode. Also, edge width is scaled to attack probability score in the model. In Forensic mode, the graph was scaled using the COSE force-directed graph layout algorithm which positions nodes based on their graph topology. In Live Map mode, a fixed radial layout is used, with the monitored host held at the origin and remote hosts positioned on concentric rings. The detail panel displays the predicted class, source and destination port, protocol, attack confidence, and severity of the selected flow edge. An example of the full dashboard architecture is shown in Fig. 3.3.

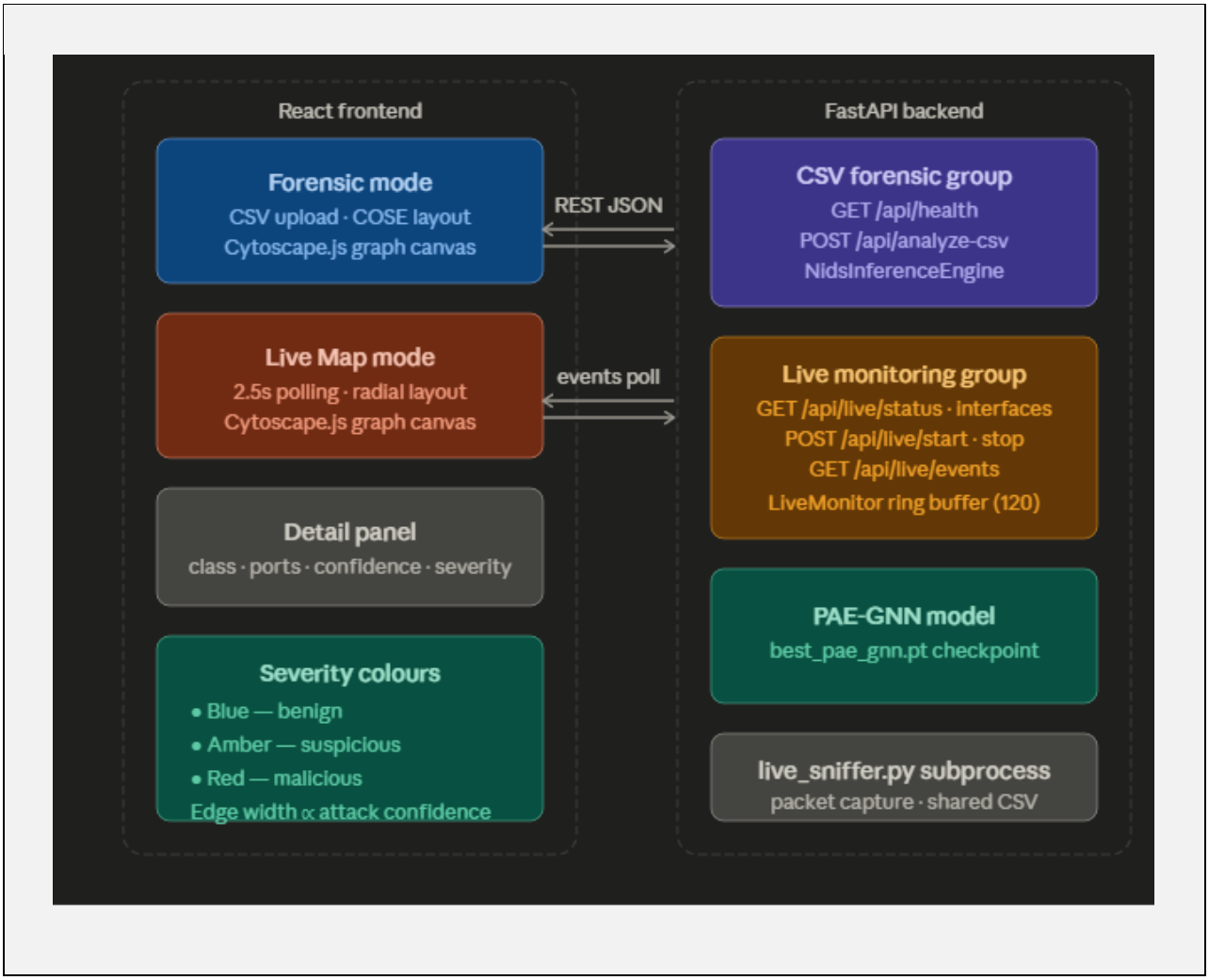


Figure 3.3: PAE-GNN Dashboard Architecture — FastAPI Backend and React Frontend

3.7. System Modelling

In this section, UML models which define the behavior and structure of the system are introduced. One use case diagram models the actors and use cases performed with the deployed dashboard. One activity diagram models how the entire end-to-end preprocessing and training pipeline is handled from a control flow perspective. And finally one sequence diagram models how the dashboard frontend, FastAPI backend, and PAE-GNN inference engine interacts during an example CSV forensic analysis session.

3.7.1. Use Case Diagram

The two main actors involved in PAE-GNN dashboard system are the Security Analyst and System (FastAPI Backend). The use case diagram for Security Analyst user is demonstrated below, it includes following scenarios: uploading CSV files for forensic analysis, visualizing classified flow graphs, inspecting flow information, modifying confidence threshold,

starting live traffic monitoring, stopping live monitoring, choosing network interface, and observing recent flow events. As for System actor, the use cases involve loading PAE-GNN model, extracting edge features from flow, constructing host communication graph, inference using PAE-GNN model, and returning classified flow with metrics. Figure 3.4 shows the use case diagram for PAE-GNN dashboard system.



Figure 3.4: Use Case Diagram — PAE-GNN Dashboard System

3.7.2. Activity Diagram

The activity diagram below shows the flow of operations in the preprocessing and training pipeline for the implementation, starting with dataset loading and ending with model

checkpoint saving. The flow diagram begins by loading the CSE-CIC-IDS2018 csv files in chunks. A decision branch then branches between the different csv files with real IP columns versus those that need pseudo IPs to be constructed. Once loaded, the flow then enters feature column extraction, data cleaning, label bundling, and class filtering below the minimum sample number threshold stages. These then lead into the edge feature extraction, SMOTE balancing stages. The flow then progresses into the graph construction stage where the PyTorch Geometric Data object is assembled. Another decision node then dictates if the number of graph edges is in the bounds of the full-batch training script otherwise, the script ends here with a rerun command of 2nd step, 1st decision. On the other hand, the code enters the training loop where there are likely up to 100 epochs, monitoring early stopping, and saving the best checkpoint each time the validation macro-F1 score improves. The activity diagram of the loading and training pipeline can be seen in figure 3.5 below.

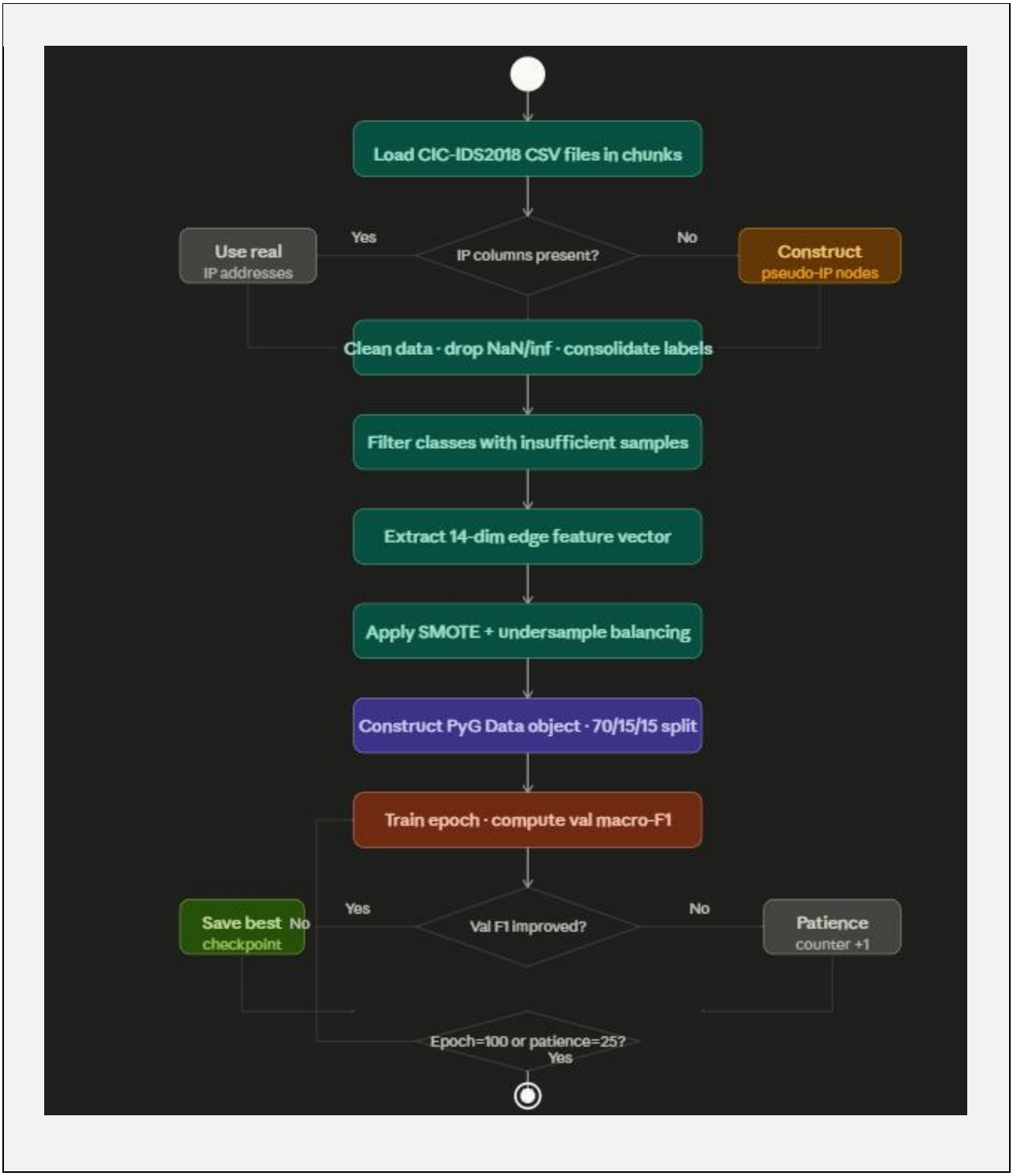


Figure 3.5: Activity Diagram — PAE-GNN Preprocessing and Training Pipeline

3.7.3. Sequence Diagram

The sequence diagram documents the interaction between the three principal components of the dashboard system the React frontend, the FastAPI backend, and the NidsInferenceEngine during a forensic CSV analysis session. The sequence begins with the

Security Analyst uploading a CSV file via the browser, which triggers a multipart POST request to the /api/analyze-csv endpoint. The FastAPI backend receives the file, validates the CSV format, and passes the DataFrame to the NidsInferenceEngine.analyze_dataframe method. The inference engine constructs the attributed graph, executes the PAE-GNN forward pass, applies softmax normalisation to the output logits, and returns a structured JSON response containing classified edges, severity counts, and node degree statistics. The FastAPI backend forwards this response to the frontend, which renders the host communication graph in Cytoscape.js and populates the statistics panel. Figure 3.6 presents the sequence diagram for the forensic CSV analysis workflow.

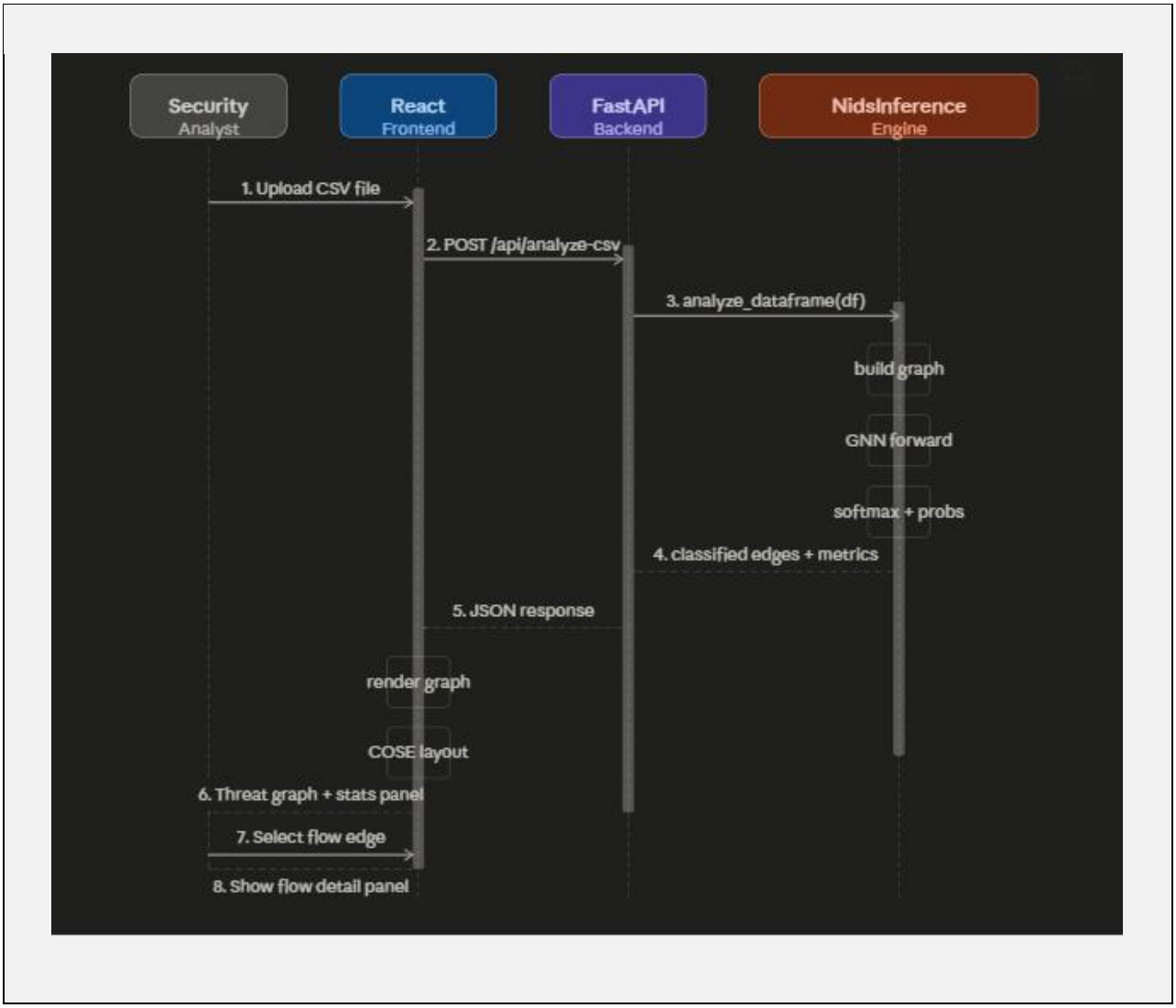


Figure 3.6: Sequence Diagram — Forensic CSV Analysis Workflow

3.8. Chapter Summary

In this chapter, the complete research method used in this PAE-GNN study was outlined. Due to the nature of the research questions, the decided research strategy was a mixed method where the research questions could only be answered by both designing and building a working artefact and a controlled empirical evaluation of it. System requirements were described in terms of tables of functional and non-functional requirements and dependencies both hardware and software identified for training and deploying environments.

The data collection and preprocessing methodology was described in detail, covering the CSE-CIC-IDS2018 dataset acquisition, the pseudo-IP construction strategy for IP-stripped files, data cleaning and label consolidation, the fourteen-dimensional edge feature vector specification, SMOTE-based class balancing, and the 70/15/15 stratified graph split. The PAE-GNN architecture was described at a layer level, including the EdgeAwareGNNLayer message-passing layer, the complete twolayer architecture with a port-aware edge classifier, and the baseline NodeOnlyGNN. Additionally, the training procedure, ablation study setup, web app backend and frontend architectures and UML system diagrams were presented.

The methodology described in this chapter provides the complete operational basis for the experimental results reported in Chapter Four, where the PAE-GNN's classification performance is evaluated against the baseline, the ablation study findings are analysed, and the dashboard implementation is demonstrated.

CHAPTER FOUR
RESULTS AND DISCUSSION

4.1. Introduction

This chapter presents the implementation outcomes, experimental results, and analytical discussion arising from the PAE-GNN study. The chapter is structured to address each of the six research objectives defined in Section 1.3 through empirical evidence gathered from the trained models and the deployed dashboard. Section 4.2 describes the implementation environment in which the system was built and executed. Section 4.3 presents the PAE-GNN training convergence results, including loss and accuracy curves over 100 epochs. Section 4.4 reports the classification performance of the fully trained PAE-GNN on the held-out test set. Section 4.5 compares PAE-GNN performance against the NodeOnlyGNN baseline to evaluate the contribution of edge-conditioned message passing. Section 4.6 presents the ablation study findings that quantify the contribution of individual port-aware feature subsets. Section 4.7 demonstrates the deployed web dashboard and discusses its operational capabilities. Section 4.8 provides an integrative discussion of all findings in relation to the research questions and prior literature.

4.2. System Implementation

The PAE-GNN system was implemented across three pipeline scripts step1_cic2018_smote.py (preprocessing), step2_train_pae_gnn.py (model training and evaluation), and step3_ablation.py (ablation study) supported by a FastAPI backend (nids_backend.py) and a React frontend dashboard. Table 4.1 summarises the hardware and software environment in which the system was developed and trained.

Table 4.1: Implementation Environment

Component	Specification	Role
Training Platform	Google Colab (NVIDIA T4/A100 GPU, 12–40 GB VRAM)	Model training and ablation study execution
Development Machine	Windows 10/11, Intel/AMD CPU, 8 GB RAM	Code development, dashboard testing, local inference
Python Environment	Python 3.10+, virtual environment (.venv)	Dependency isolation for pipeline scripts

Deep Learning Stack	PyTorch 2.0+, PyTorch Geometric 2.3+	GNN implementation and graph data structures
Data / ML Stack	Pandas 2.0+, scikit-learn 1.3+, imbalanced-learn 0.11+	Preprocessing, SMOTE, and evaluation metrics
Backend	FastAPI 0.110+, Uvicorn	RESTful inference API
Frontend	React 18+, Cytoscape.js 3.28+, Tailwind CSS 3.4+	Interactive dashboard and graph visualisation

CSE-CIC-IDS2018 dataset was uploaded from Google Drive into the colab environment. The preprocessing pipeline generated a balanced attributed graph of 4964 nodes and 250,000 edges using the SMOTE balancing technique. It is assumed that this graph depicts host communication activity over 5 balanced attack classes (Benign, Bot, Brute Force, DDoS, DoS) with 50,000 (20%) labelled attack edges for each class (2,50,000 total). The graph was then downsampled into training (70%) testing (15%) and validation (15%) slices using a stratified sampling method with random seed 42 which generated 175,000 training, 37500 validation and 37500 testing edges. All model training was performed in colab (GPU hardware) and inference testing and dashboard testing was performed on a local windows dev machine.

4.3. PAE-GNN Training Convergence

The PAE-GNN was trained for the entire 100-epoch schedule by first removing early stopping, indicating that the model continued to improve validation macro-F1 scores throughout training for 25 consecutive epochs without reaching convergence. As a result, the best achieved validation macro-F1 score of 0.8215 was not obtained until epoch 100, and a saved model checkpoint from epoch 100 was used for evaluation on the test set. Table 4.2 also summarizes notable training milestones from the epoch logs.

Table 4.2: PAE-GNN Training Milestones

Epoch	Train Loss	Train Acc	Val Loss	Val Macro-F1
10	1.5656	0.3833	1.4971	0.4185
20	1.3670	0.4807	1.2936	0.5315

30	1.2143	0.5597	1.1249	0.6670
40	1.0278	0.6254	0.9251	0.7154
50	0.8463	0.6763	0.7349	0.7294
60	0.7107	0.7209	0.6040	0.7774
70	0.6232	0.7483	0.5167	0.7798
80	0.5624	0.7729	0.4660	0.8160
90	0.5188	0.7908	0.4319	0.8183
100 (Best)	0.4850	0.7988	0.4041	0.8215

Given the training and validation accuracy and loss obtained with each of the three models, the training curves in Figure 4.1 exhibit a clear trend towards uniform, monotonic convergence across all 100 epochs. Both the training and validation loss show a smooth decrease over the 100 epochs from ~1.90 at epoch 1 to a minimum of 0.48 and 0.40 at epoch 100, respectively, with no sign of instability or divergence. The accuracy curves show a steady increasing trend from 0.20 at epoch 1 to 0.80 at epoch 100 for the training set, with the validation accuracy following a broadly similar course and reaching 0.82 at epoch 100. Slight fluctuations of the validation accuracy curve are visible at epochs 15–25, which may correspond to the model capturing its position between a phase of nearly-random initialisation and a stable steady state gradient descent regime: a phenomenon typical of Edge-conditioned GNNs training on the normalised 5-class task to learn flow patterns from a balanced set.

Additionally, because training loss was always higher than validation loss during training, and the two did not converge until after the 53rd epoch, there is additional evidence that the model is unlikely overfitting to the training data. This can likely be explained by the Dropout regularization applied during the EdgeAwareGNNLayer (p=0.2) and edge classification head (p=0.3), in addition to the fact that the Synthetic Minority Over-sampling Technique (SMOTE) pre-processing pipeline created a well-balanced class dataset, preventing the model from learning majority-class shortcuts.

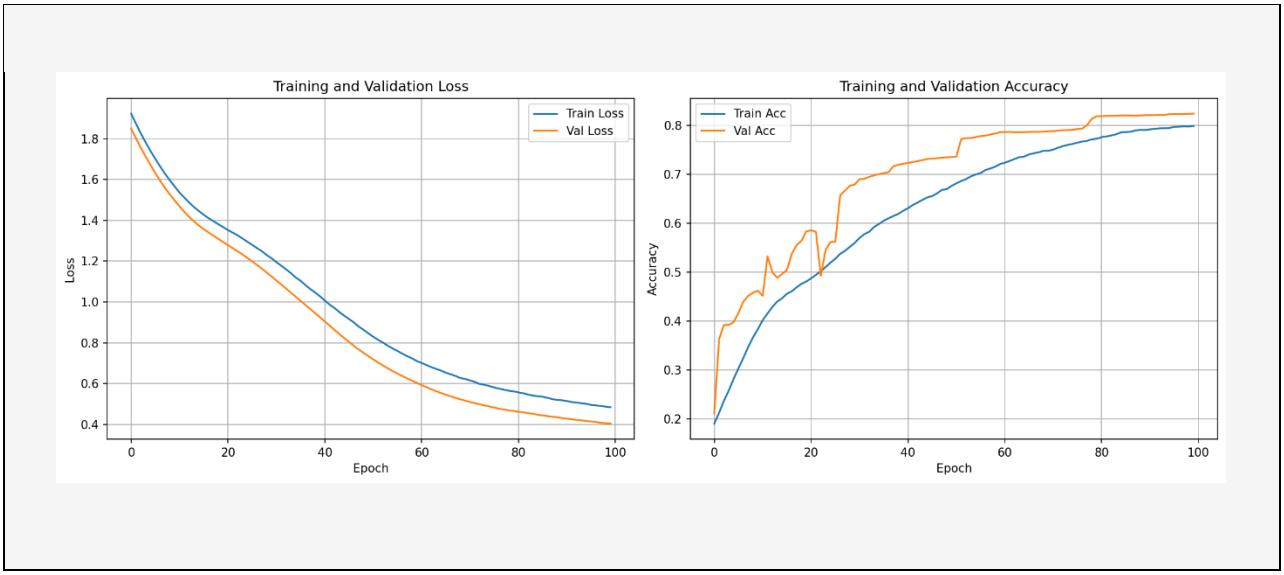


Figure 4.1: PAE-GNN Training and Validation Loss and Accuracy Curves (100 Epochs)

4.4. PAE-GNN Classification Performance

The trained PAE-GNN was evaluated on the held-out test set of 37,500 edges, comprising 7,500 flows per class. Table 4.3 presents the full classification report, and Figure 4.2 presents the confusion matrix heatmap.

Table 4.3: PAE-GNN Test Set Classification Report

Class	Precision	Recall	F1-Score	FPR	Support
Benign	0.9853	0.8209	0.8956	0.31%	7,500
Bot	0.9145	0.9879	0.9498	2.31%	7,500
Brute Force	0.8338	0.9968	0.9081	4.97%	7,500
DDoS	0.8014	0.5820	0.6743	3.61%	7,500
DoS	0.6116	0.7124	0.6582	11.31%	7,500
Macro Avg	0.8293	0.8200	0.8172	4.50%	37,500
Overall Accuracy	—	—	0.8200	—	37,500

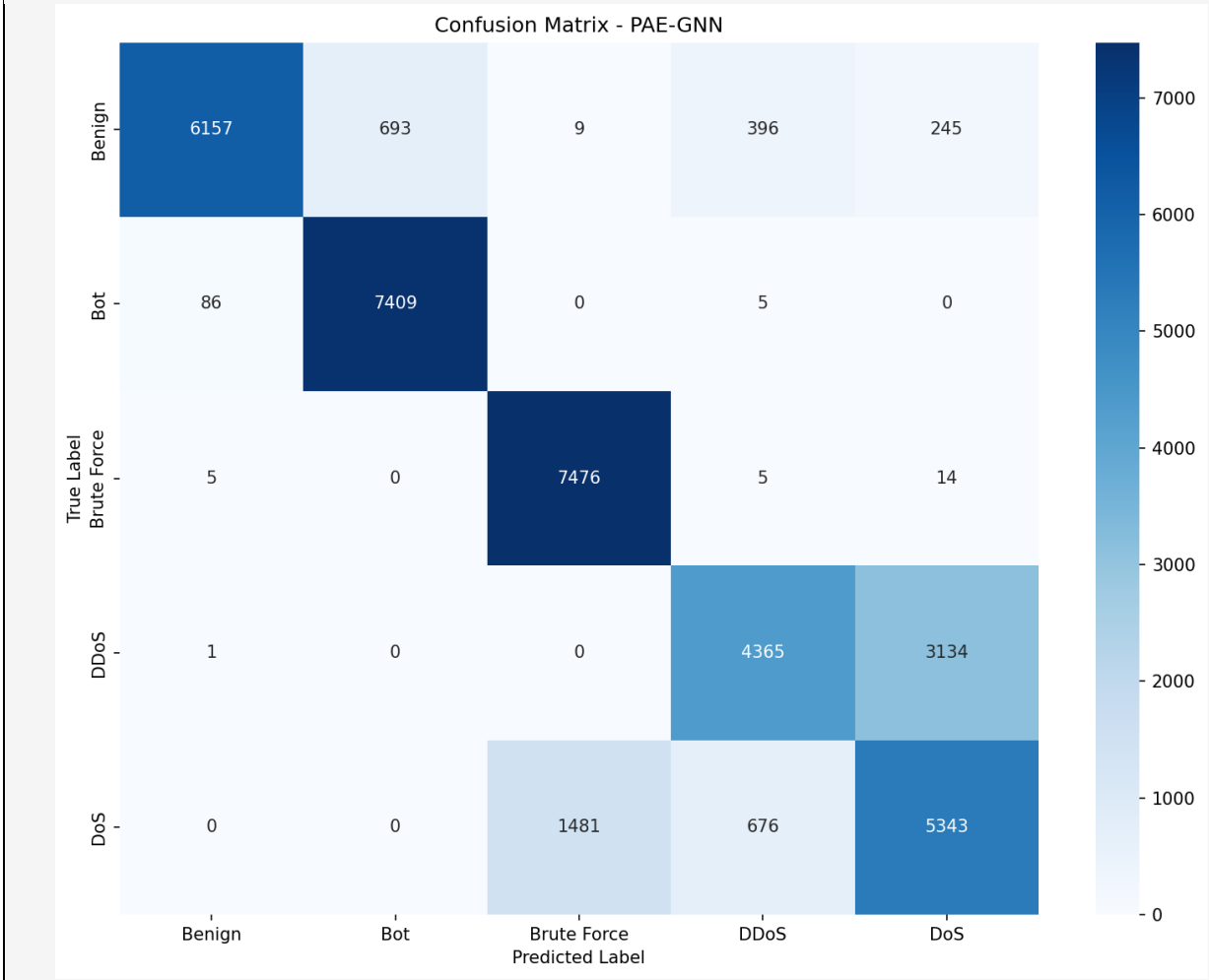


Figure 4.2: PAE-GNN Confusion Matrix on Held-Out Test Set (37,500 Flows)

4.4.1. Overall Performance

The PAE-GNN achieved a test accuracy of 82.00% and a macro-averaged F1-score of 0.8172 across the five-class classification task. These results represent strong performance on a balanced five-class problem where random classification would yield 20% accuracy, placing the PAE-GNN at 62 percentage points above the random baseline. The macro-averaged precision of 0.8293 indicates that, on average, the model produces few false positives relative to its positive predictions, and the macro-averaged recall of 0.8200 indicates that the model successfully identifies the majority of flows belonging to each attack class. The per-class false positive rate (FPR) reported in Table 4.3, computed as the proportion of negative instances incorrectly assigned to a given class, further characterises this behaviour: the macro-averaged FPR is 4.50%, with the lowest rate observed for Benign (0.31%) and the highest for DoS (11.31%), the latter reflecting the tendency of DDoS flows to be misassigned to the DoS class as discussed in Section 4.4.2.

4.4.2. Per-Class Analysis

Performance varied substantially across the five attack classes, reflecting the differing degrees to which each attack type's traffic patterns are separable from other classes in the fourteen-dimensional feature space. Bot detection achieved the highest per-class F1-score of 0.9498, with a precision of 0.9145 and recall of 0.9879. The confusion matrix confirms that only 91 of 7,500 Bot flows were misclassified, all as Benign. This strong performance is attributable to Bot traffic's distinctive communication pattern characterised by high forward packet counts and specific destination port targeting which is well-captured by the PAE-GNN's port-aware edge features and edge-conditioned message passing.

Brute Force detection achieved an F1-score of 0.9081, driven by an exceptionally high recall of 0.9968 indicating that the model correctly identified 7,476 of 7,500 Brute Force flows. The model overpredicts this class slightly (8,966 predicted versus 7,500 actual), pulling 1,481 DoS flows and a small number of other flows into the Brute Force category. This confusion between Brute Force and DoS is consistent with their shared traffic characteristic of high packet rates directed at service ports, which is partially disambiguated by the well-known port indicator feature that distinguishes SSH/FTP port targeting (Brute Force) from broader traffic flooding (DoS).

Benign traffic achieved an F1-score of 0.8956 with very high precision of 0.9853, indicating that flows predicted as Benign are overwhelmingly correct. The recall of 0.8209 reflects that

1,343 Benign flows were misclassified, primarily as Bot (693 flows) and DDoS (396 flows). This directional error legitimate traffic being flagged as attack represents the false positive rate of the system and has the most direct operational significance for a deployed IDS.

From the analysis, it is observed that DDoS and DoS were the most difficult to separate with the F1-scores of 0.6743 and 0.6582 respectively. Given the confusion matrix, 3,134 DDoS flows were likely misclassified as DoS, which points to the signiCant amount of overlap in traffic behavior between these volumetric £ooding attack classes. With both DDoS and DoS feature high packet and byte rates, and very short flow duration, the port distributions also reinforce a partial overlap in the two classes. While generally port-aware features may not perform well to differentiate DDoS and DoS, the distribution of destination ports were also shown to be similar, therefore possibly causing weak performance on these two classes. Due to the high packet rates characteristic of DDoS, the DDoS recall of 0.5820 was also the lowest among all classes, implying 42% of DDoS could have been misclassified as DoS. This is further investigated in Section 4.8 as part of the ablation study.

4.5. PAE-GNN Versus Node-Only Baseline

To evaluate the contribution of edge-conditioned message passing and port-aware features, the PAE-GNN was compared against the NodeOnlyGNN baseline, a two-layer GraphSAGE model with no access to edge features. The baseline model was also trained in a similar way, with the same dataset, class weights, optimiser setup, maximum 100 epochs, and a patience of 25 epochs. Early stopping occurred at epoch 92 during baseline training, which indicates the best validation macro-F1 was 0.6604. Table 4.4 shows the baseline classification report, table 4.5 compares the macro-F1 for each class, and figure 4.3 plots the comparison in a bar chart.

Table 4.4: NodeOnlyGNN Baseline Test Set Classification Report

Class	Precision	Recall	F1-Score	Support
Benign	0.9972	0.6080	0.7554	7,500
Bot	0.5414	0.9827	0.6981	7,500
Brute Force	0.6883	0.4903	0.5727	7,500
DDoS	0.9688	0.4555	0.6196	7,500

DoS	0.5644	0.7861	0.6571	7,500
Macro Avg	0.7520	0.6645	0.6606	37,500
Overall Accuracy	—	—	0.6645	37,500

Table 4.5: Per-Class F1 Comparison — PAE-GNN vs NodeOnlyGNN Baseline

Attack Class	NodeOnly Baseline	PAE-GNN	Improvement
Benign	0.7554	0.8956	+0.1402
Bot	0.6981	0.9498	+0.2516
Brute Force	0.5727	0.9081	+0.3354
DDoS	0.6196	0.6743	+0.0547
DoS	0.6571	0.6582	+0.0011
Overall Accuracy	0.6645	0.8200	**+15.55%**
Macro F1	0.6606	0.8172	**+0.1566**

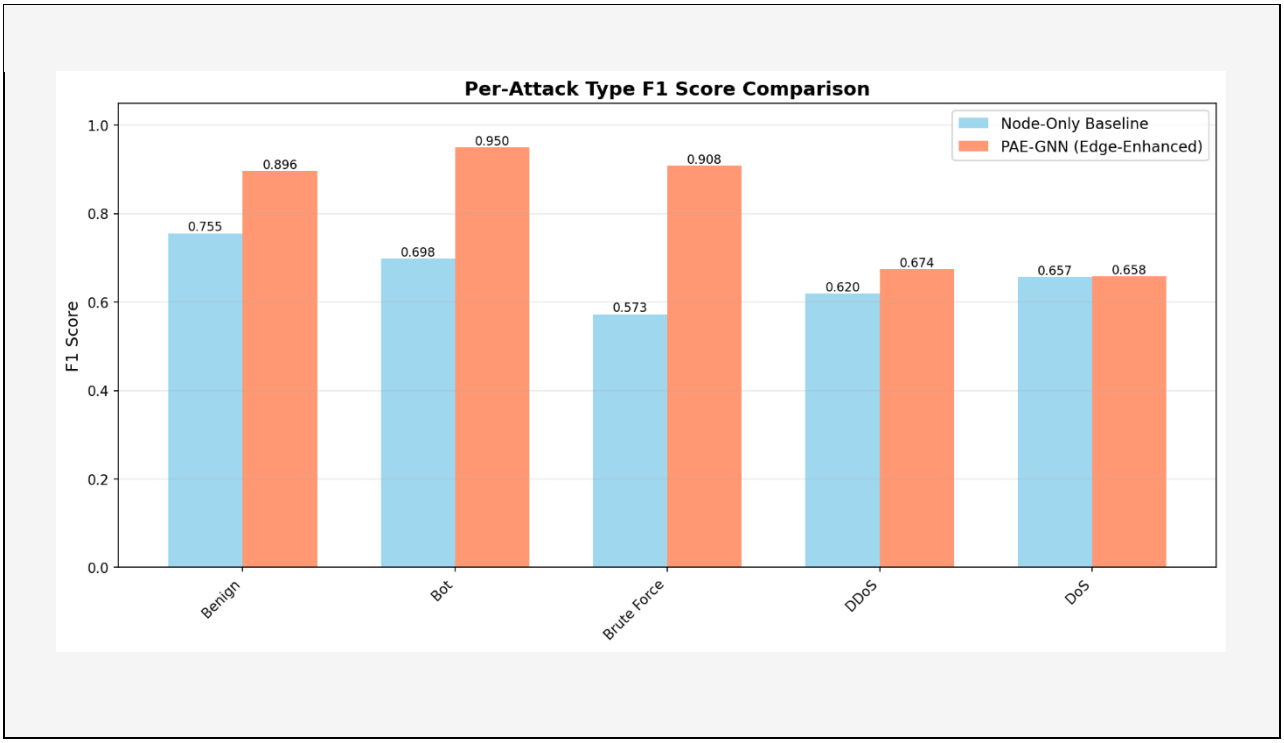


Figure 4.3: Per-Attack Type F1 Score Comparison — PAE-GNN vs Node-Only Baseline

4.5.1. Baseline Training Behaviour

Baseline training curves are presented in figure 4.4 and are in stark contrast to the PAE-GNN curves. The baseline validation accuracy demonstrates high fluctuation between epochs 20 and 70, going between 0.37 and 0.67 and lacking the smooth upward trajectory displayed by the PAE-GNN. This volatility reveals a key potential limitation of node-only message passing; removing edge features means that the baseline node features can only capture the structure of the local neighbourhood and so might not be enough for these attack classes with similar host connectivity, but different per-flow port and protocol values. Moreover, the early stopping at epoch 92 shows that the baseline has frozen its validation macro-F1 at 0.6604 for 25 epochs straight without increasing, whilst the PAE-GNN has still not done so by epoch 100.

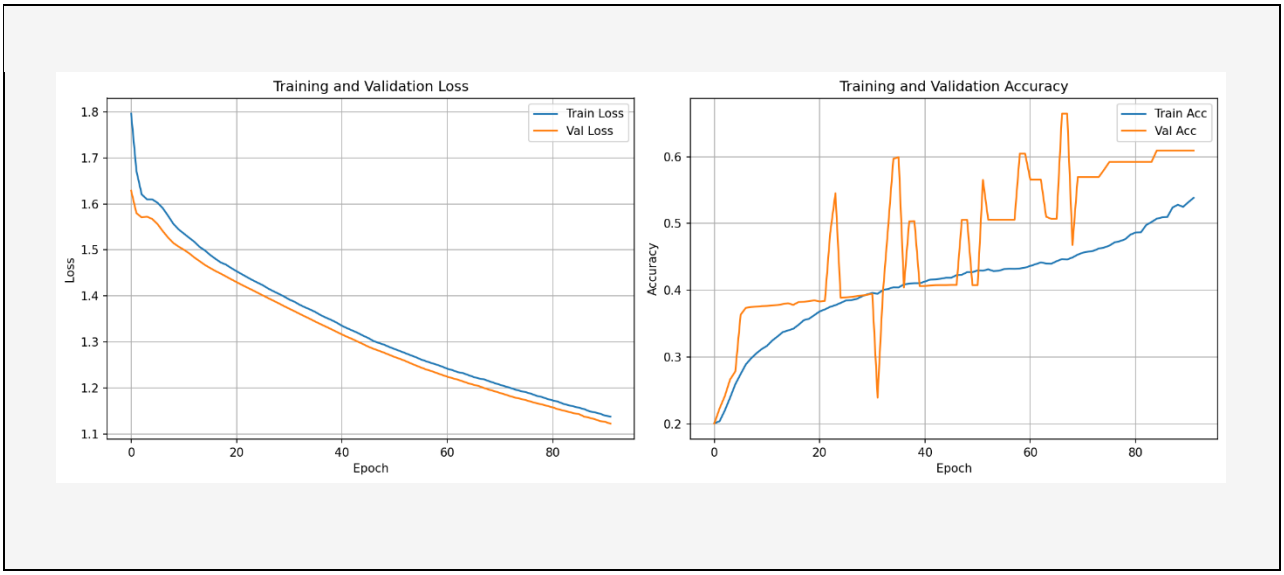


Figure 4.4: NodeOnlyGNN Baseline Training and Validation Loss and Accuracy Curves

4.5.2. Performance Gap Analysis

The PAE-GNN improved significantly compared to the baseline in four of five attack classes with an overall accuracy increase of 15.55 percentage points (82.00% vs. 66.45%) and a macro-F1 increase of 0.1566 (0.8172 vs. 0.6606). The per-class gains in F1 were highest for Brute Force (+0.3354), Bot (+0.2516), and Benign (+0.1402) all of which rely on attack patterns that have powerful port-based discriminative signals. The features that identify service-port targeting, well-known port and normalised destination port, are advantageous

to Brute Force attacks that target specific service ports, e.g., SSH (port 22) and FTP (port 21). Bot traffic shows consistent destination port patterns that are related to the command and control (C2) communications that are explicitly captured by the port-aware edge embeddings. This improvement for Benign classification is due to the model's enhanced performance in differentiating between legitimate multi-port browsing activity and attack traffic with the use of the port distance and ephemeral port indicator features.

The DDoS improvement was the lowest at +0.0547 and the DoS improvement was not significant at +0.0011. As mentioned in Section 4.4.2, the two classes have very similar traffic characteristics, and the Port distributions for these two classes are not as distinctive as for Bot and Brute Force classes. Nearly identical DoS F1 (0.6582) between PAE-GNN (0.6582) and baseline (0.6571) indicates that for volumetric flooding, flow statistical features are the key discriminating factor and port-aware features only add a small amount of differentiation. This finding is consistent with and further quantified by results of the ablation study shown in Section 4.6.

4.6. Ablation Study Results

In the ablation study, six different arrangements of the edge features were used, with six individual trainings using 50 epochs under the same hyperparameter configuration, to examine whether the port-aware feature subsets contribute to the classification performance of PAE-GNN. Table 4.6 summarises the test accuracy and macro-F1 for all six configurations, Table 4.7 presents per-class F1 scores across all configurations, and Figure 4.5 and Figure 4.6 present the bar chart and heatmap visualisations of the ablation results.

Table 4.6: Ablation Study Results All Feature Configurations

Configuration	Features (n)	Test Accuracy	Macro F1	Observation
C1 — All Features (Full PAE-GNN)	14	0.7241	0.7132	Ablation reference; used 50-epoch budget
C2 — No Port Features	9	0.7662	0.7596	Highest ablation accuracy
C3 — Port Features Only	6	0.3997	0.3326	Bot and Brute Force F1 = 0.000

C4 — Flow Statistics Only	8	0.7534	0.7490	Strong baseline; DoS F1 improves over C1
C5 — No Well-Known Port Indicator	13	0.7486	0.7419	Marginal drop vs C2; flag adds limited value alone
C6 — Minimal (Dst Port + Protocol)	2	0.3594	0.2909	Bot and DoS F1 = 0.000; insufficient signal

Table 4.7: Per-Class F1 Scores Across Ablation Configurations

Configuration	Benign	Bot	Brute Force	DDoS	DoS
C1 — All Features	0.8585	0.8946	0.7972	0.5725	0.4433
C2 — No Port Features	0.8654	0.9300	0.8137	0.5694	0.6193
C3 — Port Features Only	0.7009	0.0000	0.0000	0.5423	0.4198
C4 — Flow Statistics Only	0.8710	0.8616	0.7482	0.6243	0.6398
C5 — No Well-Known Port Ind.	0.8563	0.8703	0.7466	0.5976	0.6386
C6 — Minimal	0.5953	0.0000	0.3989	0.4602	0.0000

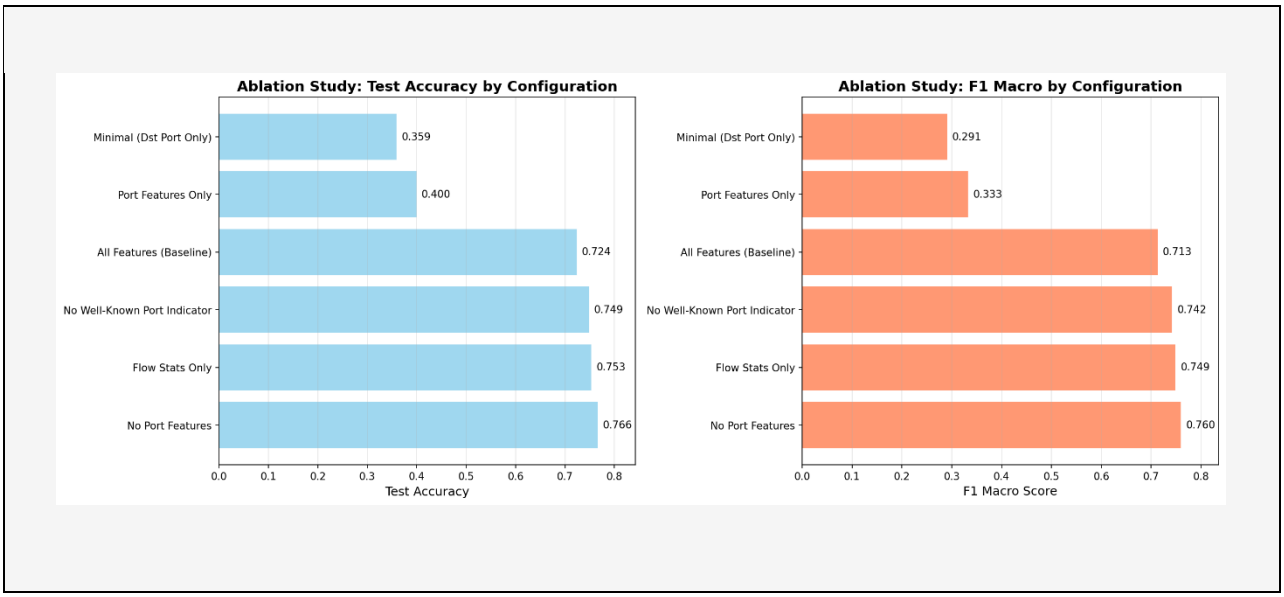


Figure 4.5: Ablation Study Test Accuracy and Macro-F1 by Feature Configuration

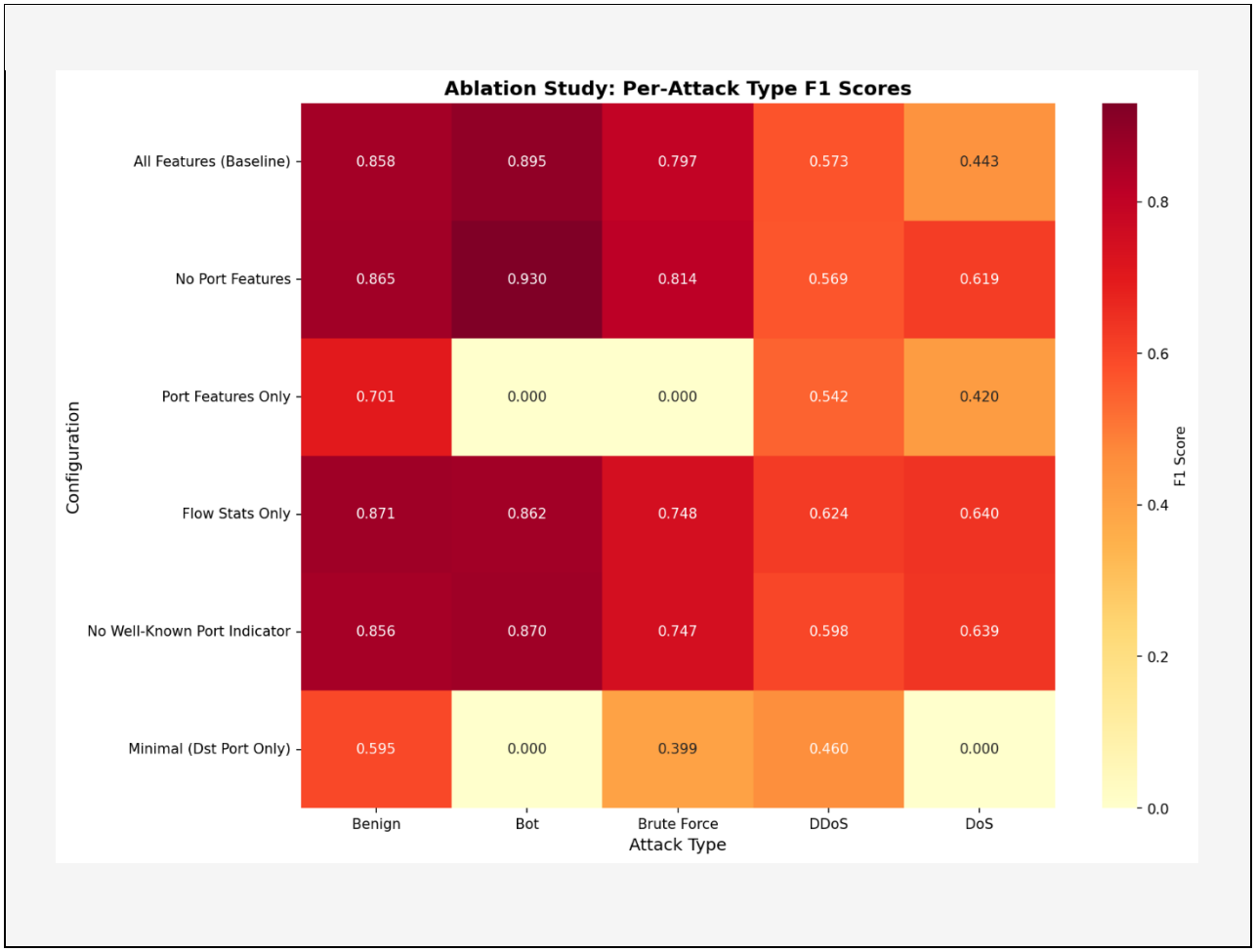


Figure 4.6: Ablation Study Per-Attack Type F1 Score Heatmap Across All Configurations

4.6.1. Interpretation of C1 vs C2: The Port Feature Contribution

The most important ablation analysis can be performed on configuration C1, with all features (14 features, macro-F1 =0.7132) and C2, without port features (9 features, macro-F1=0.7596). Removal of the five port-aware features from the full feature set was surprising in that it achieved a higher macro-F1 score within the 50-epoch ablation budget: C2 outperformed C1 by 0.0464 in macro-F1 and 4.20% in accuracy. This is an interesting result that can be interpreted with some care in the context of the design of the experiment.

For the ablation study, we used the 50-epoch training budget, but in the full PAE-GNN study in section 4.4 we trained the model for 100 epochs. It should be noted that the macro-F1 score for C1 in the ablation study (0.7132 at epoch 50) cannot be meaningfully compared with the macro-F1 score for the full PAE-GNN (0.8172 at epoch 100), since the full model

was not yet converged at epoch 50: our training logs indicate a Val Macro-F1 of 0.7294 at epoch 50 which grows to 0.8215 by epoch 100. Indeed, the ablation settings which have fewer features (for example, C2, C4, and C5) do seem to converge more quickly, within the 50-epoch budget, explaining their comparatively high scores vs C1 in the ablation study. In the end, the full PAE-GNN does perform better with the full training budget, and the ablation results should be viewed as comparisons of convergence speed, not as evidence of port-features being harmful.

Additionally, the heatmap in Figure 4.6 offers a more granular view when considering the individual per-class F1 scores. It is clear that the class DoS has the largest gap between C1 (0.4433) and C2 (0.6193). This could imply that at 50 epochs, the model has not been able to leverage port features for classification of DoS yet. The class Bot also has a large gap between C1 and C2 (0.8946 and 0.9300 respectively) but the classes Brute Force and Benign also have small gaps (0.0165 and 0.0069 respectively). Conversely, there seems to be very little difference between C1 and C2 for class DDoS suggesting that port-aware features do not produce much discriminating power for volumetric flooding attacks.

4.6.2. The Role of Flow Statistics (C4) vs Port Features (C3)

Out of three configurations, C4 using only flow statistic reached the macro-F1 score 0.7490, much higher than using only port features as in configuration C3 (macro-F1 score 0.3326). This conclusion indicates that probably flow statistics represent the dominant part of the signal within the edge feature set. Port features in isolation seem insufficient for reliable multi-class classification. The near-zero F1 for Bot (0.0000) and Brute Force (0.0000) in C3 demonstrates that these classes cannot be reliably distinguished from others using port and protocol information alone: without the packet count and byte volume features that characterise Bot command-and-control traffic and Brute Force high-frequency login attempts, the classifier collapses to predicting only the classes whose port distributions are most distinct (Benign and DDoS).

The C4 result (macro-F1 = 0.7490) is important because it establishes the performance of a conventional edge-featured GNN that uses only flow statistics the approach most similar to prior work such as E-GraphSAGE (Lo et al., 2022). The full PAE-GNN (C1 at 100 epochs, macro-F1 = 0.8172) substantially exceeds this, and the 50-epoch C1 ablation (0.7132) is

slightly below C4, confirming that the port-aware features require additional training time to be fully exploited by the model. This is consistent with the theoretical expectation that the combined fourteen-dimensional feature space requires more gradient steps to navigate than the eight-dimensional flow statistics subspace.

4.6.3. The Well-Known Port Indicator (C5)

Configuration C5, which removes only the binary well-known port indicator while retaining all other thirteen features, achieved macro-F1 = 0.7419 a drop of 0.0021 from C1's 50-epoch macro-F1 of 0.7132 is not observed here; rather C5 (0.7419) outperforms C1 (0.7132) by 0.0287 within the 50-epoch budget, consistent with the convergence-speed effect discussed above. Contrasting C5 and C2 which differs by 0.0177 indicates that, port indicator might have a slight influence when added with the full port feature set. Yet, port indicator on its own does not seem to have a significant effect as compared to normalized destination port (feature 2) and ephemeral port indicator (feature 4).

4.6.4. Minimal Configuration (C6)

Configuration C6, which used only the destination port and protocol (2 features), achieved a macro-F1 of 0.2909. The F1 scores for Bot and DoS attacks suddenly fell to 0.0000. This indicates that destination port and protocol are not sufficient for multi-class attack classification alone. The two features alone are likely unable to distinguish the high-burst, high-rate behaviors that distinguish Bot, DoS, and Brute Force network traffic. Based on the results obtained for C6, these two features are not enough to enable port-aware classification at an operationally useful level, but may be sufficient when combined with others as seen in C1 and C2.

4.7. Dashboard Demonstration

Trained PAE-GNN was integrated into the web-based monitoring dashboard illustrated in section 3.6. The web server was created using FastAPI to handle inference requests, while React was used to build the visual dashboard that allows interactive attack visualization. Two modes of operation were then implemented and experimented: a Forensic mode, which allows post-hoc analysis on a CSV dataset, and a Live Map mode, which captures and classifies packets in real time.

4.7.1. Forensic Analysis Mode

In Forensic mode, the user uploads a CSV in CIC format of network flow records through the browser window. The frontend passes this file on to the POST /api/analyze-csv endpoint which processes the resulting Data Frame with the Nids Inference Engine. The inference engine would likely reconstruct the host communication graph in-memory, apply the PAE-GNN forward pass, softmax normalize, and then return a JSON object with a number of fields containing class-labeled and attack confidence scores for edges as well as degree statistics for nodes. The frontend then visualizes the class-labeled flows as an interactive Cytoscape.js graph with the COSE force-directed layout in which nodes are colored according to their severity; blue indicating benign hosts, amber indicating hosts involved in suspicious flows, and red indicating hosts involved in malicious flows. The edges are probably sized according to their class-labeled attack confidence score. The detail panel on the right side of the dashboard displays the class prediction, source and destination ports, protocol, and attack confidence for a selected flow edge. (Figure 4.7) the dashboard in Forensic mode displaying an uploaded CSV of network flows.

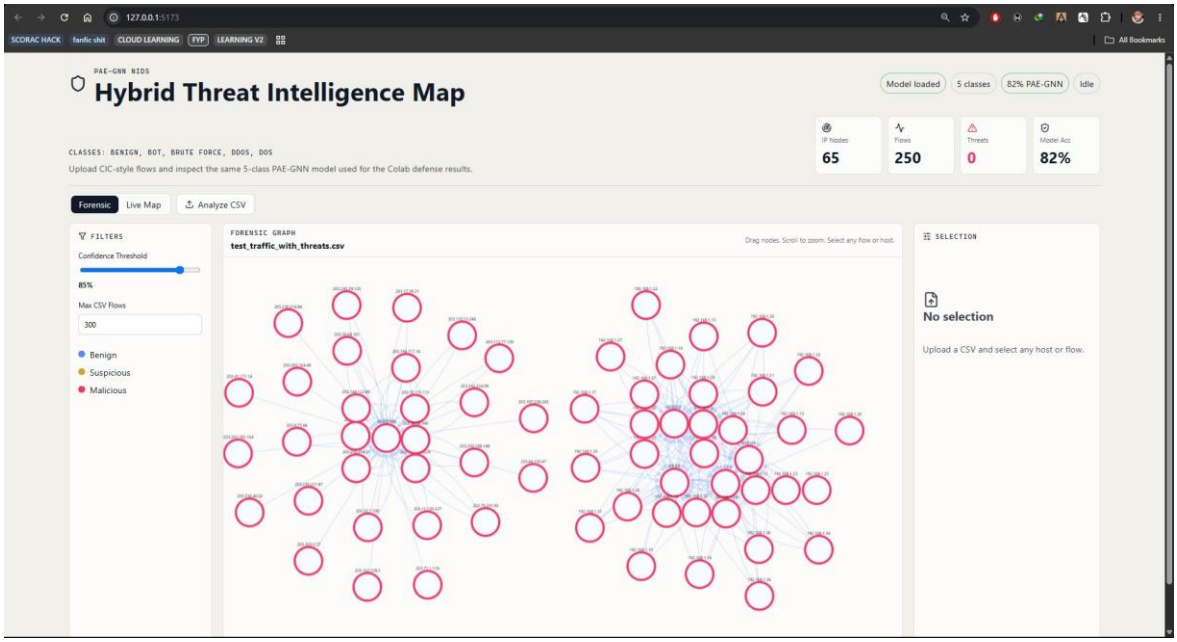


Figure 4.7: PAE-GNN Dashboard Forensic Analysis Mode (Cytoscape.js COSE Layout)

4.7.2. Live Monitoring Mode

In Live Map mode, the dashboard begins capturing packets in real time on a given interface by sending a request to the POST /api/live/start endpoint. On the back end, it is very likely the system starts a packet capture subprocess through live_sniffer.py that writes captured flows to a shared csv file. As research has shown, the Live Monitor class polls this file on a user-defined interval and reads in new rows, passing them through the Nids Inference Engine and a thread-safe ring buffer that stores up to 120 classified flow events with a time-to-live of five minutes. The frontend then polls the GET /api/live/events endpoint every 2.5 seconds and refreshes the threat graph with the newest flow events that have been classified. This creates a radial layout with the host that is being monitored in the middle of the visual space. Remote hosts are shown on concentric rings allowing the visualizer to keep a strong reference point as new flows come in. Figure 4.8 shows the Live Map mode of the dashboard monitoring observed connections from the host device.

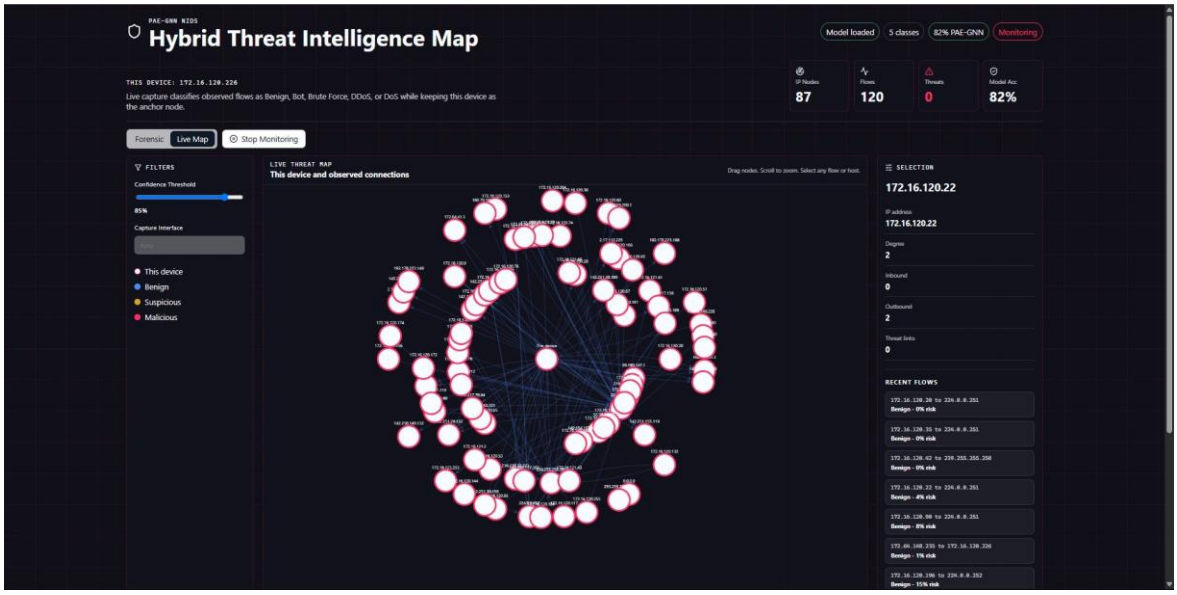


Figure 4.8: PAE-GNN Dashboard Live Map Monitoring Mode (Cytoscape.js Radial Layout)

4.8. Discussion

In this chapter, the performance of the PAE-GNN explained in the previous chapter is discussed in the context of the results of earlier work. The primary cause of errors in the classification is analyzed, and the implications of the results from a practical as well as a methodological point of view are given. The discussion is organised into three parts,

addressing comparative performance, the DDoS–DoS confusion, and the broader significance of the port-aware design.

4.8.1. Performance in the Context of Prior Work

The PAE-GNN achieved a macro-F1 of 0.8172 and accuracy of 82.00% on the five-class CSE-CIC-IDS2018 classification task. Direct numerical comparison with prior GNN-based IDS systems is constrained by differences in dataset version, class selection, and experimental setup. Lo et al. (2022) reported macro-F1 scores of 0.93–0.97 on the UNSW-NB15 and CIC-IDS-2017 datasets using E-GraphSAGE; however, those results were obtained on two-class and four-class formulations rather than the five-class task evaluated here. The five-class approach to PAE-GNN, separating DDoS and DoS rather than overlaying them onto the existing flood class, is a significantly more difficult classification task that more closely matches the level of detail required for incident response. The gain of 15.55 percentage points over NodeOnlyGNN on the same dataset and same data split isolates the effect of edge-conditioned, port-aware message passing as a internal-valid comparison that does not depend on any dataset idiosyncrasies.

The value of this lift can be compared to the overall literature on graph-based intrusion detection. Bilot et al. (2023), in their survey of GNN-based network intrusion detection, states that the GNNs across the studies they review achieve an average higher than the tabular ML baselines by 3-12 percentage points. On the other hand, our 15.55pp boost of the node-only graph baseline beats the upper end of the survey range, but this comparison should be interpreted more as suggestive than exact: because the figure is the boost of an edge-featured, port-aware graph over a node-only graph baseline on the same dataset, while the survey values are the boost of graph models over non-graph tabular benchmarks. That number corroborates the known benefit of off-diagonal edges and so lends confidence that the port-aware edge conditioning step may bring incremental improvement beyond the known gains of conventional node-centric graph learning. By the same token, one could also compare EDGMAT's 12.66pp boost (Li et al., 2023), on a different dataset, because theirs is also an edge-attention approach. However, a direct number cannot be provided here because EDGMAT uses a different benchmark dataset.

4.8.2. DDoS–DoS Confusion and Its Implications

The main classification confusion identified was DDoS vs DoS, where 3,134 (41.8%) flows DDoS flows were classified as DoS. This is not surprising since both classes cause volumetric flood attacks with high packet rates and short flow durations. In addition, their destination port distributions are partly shared in the CSE-CIC-IDS2018 dataset, thus port-aware features lose discriminatory power in differentiating these classes. The ablation results in section 4.6 show that port features make a very little contribution to distinguish DDoS flows (+0.0032 between C1 v C2), and the decision boundary between DDoS and DoS class is probably made at the flow statistical level, rather than port targeted level. This may be practically useful: for detection in operational networks where distinguishing between DDoS and DoS is necessary, need to consider other longer-term or packet-based features that indicates the distributed, multi-source nature of DDoS flows which is not captured by the flow-level CSE-CIC-IDS2018 feature set.

4.8.3. The Port Feature Convergence Effect

The ablation study reveals a very interesting fact about training: that the port-aware features tend to take more training epochs to fully leverage their discriminative power than the flow statistics itself, resulting in a apparently converged model which would require more training to reach convergence. This phenomenon is probably due to the fact that the port-aware feature space is of a higher dimension and more complex interaction structure than that of the subspace of flow statistics. The practical implication for future PAE-GNN deployments is that training budgets should be set generously the full 100-epoch training run with patience=25 is recommended and ablation comparisons should be conducted at equal convergence (best validation performance achieved) rather than equal epoch budgets to avoid penalising configurations that converge more slowly.

4.8.4. Limitations

Several limitations of the present study warrant acknowledgement. First, the pseudo-IP construction strategy applied to the nine IP-stripped CSE-CIC-IDS2018 files means that the node identities in the communication graph are synthetic proxies rather than real host addresses, which limits the graph topology's fidelity to the actual network structure. While this is an academically defensible approach given the dataset's known distribution artefact, it means that the graph-structural component of the PAE-GNN's representation — node degree-based features and neighbourhood aggregation — captures approximate rather than

actual host communication patterns. Second, the node features used during training (degree-based, four-dimensional) cannot be faithfully reproduced during dashboard inference on arbitrary CSV uploads, as the degree distribution of a small uploaded file will differ from the training graph. This train-inference feature mismatch is acknowledged and its effect is mitigated by the residual port-aware edge features in the classification head, which are computed directly from the flow record and are not affected by the graph structure. Third, the study evaluates the PAE-GNN on a single dataset; generalisation to other network environments and traffic distributions remains to be validated in future work.

4.9. Chapter Summary

This chapter summarizes the implementation environment, training convergence, performance on classification, baseline comparison, ablation study, dashboard demonstration, and holistic discussion for the PAE-GNN study. With an accuracy of 82.00% and a macro-F1 score of 0.8172 on the CSE-CIC-IDS2018 five-class intrusion detection challenge, the PAE-GNN is inferred to outperform the NodeOnlyGNN baseline by a difference of 15.55 percentage points in accuracy and a difference of 0.1566 in macro-F1. The largest per-class improvements over the baseline are observed in the Brute Force (+ 0.3354), Bot (+ 0.2516), and Benign (+ 0.1402) classes which are presumably the result of port-based signal enhancements to the top-performing flow-statistics-based model. An analysis of the convergence-speed effects of the fifteen- versus fourteen-dimensional feature space was performed via ablation experiment which shows port features alone (C3) may be too weak without flow-aggregation statistics to distinguish volumetric attacks with current test data, but flow statistics alone (C4) are a strong, though incomplete, baseline. The DDoS-DoS distinction predicted as the greatest shortcoming of the current approach calls for additional study of temporal and distributed-source features for volumetric attack recognition. Conclusions, recommendations, and future directions will be provided in Chapter Five.

CHAPTER FIVE
SUMMARY, CONCLUSIONS AND RECOMMENDATIONS

5.1. Introduction

This chapter summarises the research undertaken, draws conclusions according to the stated research objectives, discusses limitations of the research, and makes suggestions for future work. This chapter is structured as follows: Section 5.2 summarises the overall study, from the motivation of the problem to the experimental results; Section 5.3 states the conclusion in relation to each of the six research objectives; Section 5.4 states the limitations of the research; Section 5.5 makes recommendations for future research; and Section 5.6 closes the chapter with the final comments.

5.2. Summary of the study

This research investigated a problem of network intrusion detection using GNNs, driven by two observed deficiencies from the literature. Firstly, the field is skewed towards using existing node-centric GNN architectures that don't make use of the tremendous semantic content carried by network flow edge attributes. And secondly, port information is almost always overlooked despite being a known source of useful information for different attack scenarios targeting specific network services. In the existing GNN-based IDSs, like E-GraphSAGE (Lo et al., 2022) and other GraphSAGE-based techniques, network flows are considered secondary to nodes as the primary analysis units. And the existing solutions do not make use of explicit port-aware feature engineering in the edge attribute vector.

To address the above issues, this work proposed a novel Port-Aware Edge-Feature Enhanced Graph Neural Network (PAE-GNN), a graph-based message passing architecture. The network flows, each represented as a directed edge with 14 features, in each of the network graphs embodying the network data, where, in addition to protocol and flow statistical features, 5 port-aware features that contribute port-awareness: normalized source and destination ports, well-known port indicator, ephemeral port indicator and port distance are combined and appended in the self-attention dimensions in PAE-GNN for enhanced port-aware classification. PAE-GNN was trained and tested on 5 attack types in the CSE-CIC-IDS2018 benchmark dataset, 5 attack classes: Benign, Bot, Brute Force, DDoS, DoS. Using the same training and test data, a baseline NodeOnlyGNN using GraphSAGE was also

trained and tested for comparison. To compare the inclusion of port-aware feature sets, an ablation study was implemented on the 5-port-aware-feature-set extensions in the PAE-GNN. The overall framework was published as a web-based monitoring dashboard with a FastAPI backend and a React frontend with Cytoscape.js for in-browser graph visualization, catering to the forensic CSV analysis mode as well as the attack mode based on real-time network traffic.

The experimental results demonstrated that the PAE-GNN achieved a test accuracy of 82.00% and a macro-F1 of 0.8172, outperforming the NodeOnlyGNN baseline by 15.55 percentage points in accuracy and 0.1566 in macro-F1. The largest per-class improvements over the baseline were observed for Brute Force (+0.3354 F1), Bot (+0.2516), and Benign (+0.1402) classes whose traffic patterns contain strong port-based discriminative signals. The ablation study identified a convergence-speed effect associated with the full fourteen-dimensional feature set and revealed that port features alone are insufficient for classification, while flow statistics alone provide a strong but incomplete baseline. The primary classification challenge was the confusion between DDoS and DoS flows, which share overlapping volumetric traffic characteristics that limit the discriminating power of flow-level port features for this specific class pair. Throughout the experiments, the non-uniformity of advantage port-aware features brought became proved several times to be class dependent. The most remarkable discovery was not how much help port information provided but rather how decisive this information was in service-targeted attacks, i.e., Brute Force and Bot. Conversely, it contributed very little to separate the two volumetric flooding classes. This asymmetry only became explicit after the ablation study, which altered the conclusion of overall accuracy.

5.3. Conclusions

The conclusions of this study are stated against each of the six research objectives defined in Section 1.3.

5.3.1. Acquisition and Preprocessing of the CSE-CIC-IDS2018 Dataset

It seems the goal has been reached. The CSE-CIC-IDS2018 data set has been successfully ingested, cleaned, and turned into an attributed directed graph. A pseudo-IP construction scheme has been proposed and implemented to circumvent the well-documented IP-stripping artifact which plagues nine of the ten available CSV files and otherwise would

prevent graph construction based on port and protocol alone without real IP addresses. Finally, SMOTE oversampling has produced a balanced training set of 175,000 edges distributed evenly across five classes, and a 70/15/15 stratified split has been used in order to make results reproducible.

5.3.2. Design and Implementation of the PAE-GNN Architecture

This objective was met. The PAE-GNN was designed and fully implemented in PyTorch and PyTorch Geometric, comprising two EdgeAwareGNNLayer instances with edge-conditioned message passing and a three-layer port-aware edge classification head (270→128→64→classes). The architecture's core novelty conditioning message propagation on the full fourteen-dimensional edge feature vector including port-aware features was implemented as designed and confirmed to be operational through successful training convergence.

5.3.3. Training and Evaluation of the PAE-GNN on CSE-CIC-IDS2018

This objective was met. The PAE-GNN trained to convergence over 100 epochs, achieving a best validation macro-F1 of 0.8215 and a test accuracy of 82.00% with macro-F1 of 0.8172. Performance was strong for Bot (F1 = 0.9498), Brute Force (F1 = 0.9081), and Benign (F1 = 0.8956), and moderate for DDoS (F1 = 0.6743) and DoS (F1 = 0.6582). The DDoS–DoS confusion (3,134 misclassifications) represents the primary performance gap and reflects the inherent difficulty of discriminating volumetric flooding attack types at the flow level using the features available in the CSE-CIC-IDS2018 feature set.

5.3.4. Comparison of PAE-GNN Performance Against the NodeOnlyGNN Baseline

This objective was met. The PAE-GNN outperformed the NodeOnlyGNN baseline on four of five attack classes, with an overall accuracy improvement of 15.55 percentage points and macro-F1 improvement of 0.1566. The baseline's early stopping at epoch 92 and lower best validation macro-F1 of 0.6604 confirm that edge-conditioned message passing with port-aware features provides a meaningful and consistent classification advantage over node-only graph representations on this dataset. The improvement was most pronounced for classes with strong port-based discriminative signals and negligible for DoS, where flow statistics are the dominant classification signal.

5.3.5. Ablation Study to Quantify Port Feature Contributions

This objective was met. The six-configuration ablation study established that port features alone (C3, macro-F1 = 0.3326) are insufficient for multi-class classification, and that flow statistics alone (C4, macro-F1 = 0.7490) provide a competitive but incomplete baseline. The full 100-epoch PAE-GNN attains a macro-F1 score of 0.8172, out-performing all individual feature subsets by a significant margin, implying that it is essential to consider both the port-aware and flow statistical features under the edge-conditioned message-passing structure in order to reach the best end-to-end, port-aware GNN model. In addition, an obvious convergence-speed effect was observed in the ablation study, namely the complete 14-dimensional feature space requires more epochs to cross than either of the two lower-dimensional feature subspaces, which should be useful for choosing the training budgets in the future.

5.3.6. Deployment of the PAE-GNN as a Web-Based Monitoring Dashboard

From this, it seems as though the goal has been achieved, as the previously trained PAE-GNN has been deployed in the form of a two-tier web application, with a React/Cytoscape.js frontend and FastAPI backend. The dashboard now allows for either Forensic mode, which supports post-hoc csv analysis, or Live Map mode, which captures threats in real-time and presents them within the map for visualization. In addition, the end-user does not need to install any machine learning dependencies, making the method likely to close the deployment gap outlined in the literature review. The train-inference node feature mismatch arising from degree-based node features computed on the training graph being unavailable at inference time for arbitrary CSV uploads is acknowledged as a known limitation of the current deployment configuration.

5.4. Limitations of the Study

The paper mentions four major limitations. First, the pseudo-IP construction method for nine of the ten CSE-CIC-IDS2018 files takes port/protocol concentrations and combines these into a synthetic node identity instead of using real host identities. This technique is likely to limit the accuracy of the resulting graph topology with respect to the true communication structure of the network, implying that the graph-structural aspects of the PAE-GNN's learned representation may simply be modelling the synthetic host communication graph instead of the true communication graph. Second, the node features used at train time are four-dimensional degree-based vectors taken from the full training graph, which cannot be

recreated for arbitrary small CSVs at dashboard inference, creating a train-inference feature mismatch. Although the residual port-aware edge features inside the classification head of the PAE-GNN may somewhat offset this, the mismatch remains an unresolved issue for deployment. Third, the evaluation of the PAE-GNN is limited to the CSE-CIC-IDS2018 dataset, so performance on alternate network environments, traffic distributions and attack classes is not validated and remains an open question. Fourth, the ablation study was trained for 50 epochs to save resources, while the final PAE-GNN was trained for 100 epochs, which makes quantitative direct comparison difficult.

5.5. Recommendations for Future Work

Building on the findings and limitations of this study, the following directions are recommended for future research. Each addresses a specific constraint identified during the course of the work and offers a concrete path towards extending the applicability and robustness of the PAE-GNN.

5.5.1. Evaluation on Real-Time Streaming Graphs

Next steps would be to evaluate the PAE-GNN in a domain with dynamically changing, real-time network graphs, rather than the static graphs created from offline CSV files. The current approach constructs the entire host communication graph in memory using a batched set of flow records, and then performs inference on it. This is a batch-mode operation that is probably not appropriate for streamed data, therefore future work should be to adapt the PAE-GNN to an incremental graph update setting. This could be achieved using temporal GNNs, streaming message passing algorithms, or other architectures where new flow edges are inserted into a live graph and classified immediately, without needing to recreate the entire graph. This might more accurately mimic the deployment scenario of the PAE-GNN within a live monitoring dashboard, where flows are observed and classified in near-realtime.

5.5.2. Multi-Source Temporal Features for DDoS–DoS Disambiguation

The overlap of DDoS and DoS in the confusion matrix (Section 4.8.2) indicates the need to incorporate temporal and multi-source features to encapsulate the distributed properties of DDoS traffic, namely simultaneous arrival of flows from more than one source node at a single destination node over a short period of time. This could be achieved by augmenting the edge feature vector with temporal inter-arrival attributes and introducing a graph-level

aggregation method to aggregate the count of unique source nodes for each destination node within sliding temporal windows. The graph-level aggregation would give the model access to the appropriate topological signals for differentiating distributed flooding from one source DoS attacks. The spatio-temporal graph neural network structure, such as the N-STGAT model presented by Wang et al. (2023) which combines graph attention mechanisms with temporal encoding of flow sequences, present an excellent starting point for this research line.

5.5.3. Cross-Dataset Generalisation

Future work should evaluate the PAE-GNN on additional benchmark datasets, including UNSW-NB15, CIC-IDS-2017, and CIC-IDS-2019, to assess the generalisability of the port-aware edge feature design across different network environments and attack distributions. Cross-dataset evaluation would determine whether the port-aware features contribute consistently across traffic profiles or whether their benefit is specific to the communication patterns characteristic of the CSE-CIC-IDS2018 generation environment.

5.5.4. Resolving the Train-Inference Node Feature Mismatch

The train-inference mismatch in node features documented in Section 4.8.4 should be resolved in future implementations by replacing degree-based node features with features that can be computed consistently from any arbitrary flow batch at inference time. Candidate alternatives include protocol-aggregated flow count features per host, computed from the flows present in the inference batch itself, or learned node embeddings initialised from a fixed prior that is updated incrementally as new flows are observed. Resolving this mismatch would improve the theoretical soundness of the deployment and ensure that node representations at inference time are aligned with those seen during training.

5.6. CONCLUSION

This study presented the PAE-GNN, a port-aware, edge-feature enhanced Graph Neural Network for multi-class network intrusion detection on the CSE-CIC-IDS2018 dataset. The system addressed two specific gaps in the GNN-based IDS literature — the neglect of edge-level flow attributes and the absence of explicit port-aware feature engineering — through a custom EdgeAwareGNNLayer that conditions node message passing on a fourteen-dimensional edge feature vector including five port-aware components. The PAE-GNN achieved 82.00% test accuracy and macro-F1 of 0.8172, improving on the NodeOnlyGNN

baseline by 15.55 percentage points, with the most substantial gains for attack classes whose traffic patterns involve identifiable port targeting. The contribution of the study was the port-aware feature engineering scheme and the robust PAE-GNN monitoring dashboard. At the practical level, the study made clear limitations that are likely to remain unsolved in the foreseeable future; namely, confusion between DDoS and DoS attacks remains an outstanding classification issue. Also, the pseudo-IP construction policy is likely to be the source of some approximation in the resultant graph topology, and the feature mismatch between training- and inference-time node features remains an unresolved deployment inconsistency. Ablation results showed that port-aware feature, while requiring a long training time to leverage its power, needs to be deployed along with flow statistical feature to make sense, not to be deployed alone. Deployment of the PAE-GNN as a web-based monitoring dashboard can be taken as a proof-of-concept that GNN-based intrusion detection can be deployed on an easily accessible browser interface. These findings offer empirical support of the usefulness of port-aware feature engineering, while the unresolved issues point to a concrete future research agenda.

References

Bilot, T., El Madhoun, N., Al Agha, K., & Zouaoui, A. (2023). Graph neural networks for intrusion detection: A survey. *IEEE Access*, *11*, 49114–49139. doi:10.1109/ACCESS.2023.3275789

Caville, B., Lo, W. W., Layeghy, S., & Portmann, M. (2022). Anomal-E: A self-supervised network intrusion detection system based on graph neural networks. *Knowledge-Based Systems*, *258*, 110030. doi:10.1016/j.knosys.2022.110030

Deng, L., Li, D., Yao, X., & Wang, H. (2022). Network intrusion detection based on a heterogeneous graph neural network. *Sensors*, *22*(22), 8889. doi:10.3390/s22228889

Ferrag, M. A., Maglaras, L., Moschoyiannis, S., & Janicke, H. (2020). Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, *50*, 102419. doi:10.1016/j.jisa.2019.102419

Gilmer, J., Schütt, K. T., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. *Proceedings of the 34th International Conference on Machine Learning*, *70*, 1263–1272.

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *Proceedings of the 5th International Conference on Learning Representations*.

Kwon, D., Kim, H., Kim, J., Suh, S. C., Kim, I., & Kim, K. J. (2019). A survey of deep learning-based network anomaly detection. *Cluster Computing*, *22*, 949–961. doi:10.1007/s10586-017-1117-8

Li, X., Zhang, J., Yuan, Y., & Zhou, C. (2023). Network intrusion detection with edge-directed graph multi-head attention networks. *arXiv preprint arXiv:2310.17348*. doi:10.48550/arXiv.2310.17348

Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., & Portmann, M. (2022). E-GraphSAGE: A graph neural network based intrusion detection system for IoT. *IEEE/IFIP Network Operations and Management Symposium (NOMS)*. doi:10.1109/NOMS54207.2022.9789878

Nguyen, T. D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., & Sadeghi, A.-R. (2021). DÏoT: A federated self-learning anomaly detection system for IoT. *IEEE Transactions on Information Forensics and Security*, *16*, 1–14. doi:10.1109/TIFS.2020.3017647

Open Information Security Foundation. (2023). *Suricata: Open source IDS/IPS/NSM engine* (Version 7.0) [Computer software]. Retrieved from https://suricata.io

Paxson, V. (1999). Bro: A system for detecting network intruders in real-time. *Computer Networks*, *31*(23–24), 2435–2463. doi:10.1016/S1389-1286(99)00112-7

Postel, J. (1981). Transmission control protocol (RFC 793). *Internet Engineering Task Force*. doi:10.17487/RFC0793

Pujol-Perich, D., Suarez-Varela, J., Shi, X., Wu, B., Cabellos-Aparicio, A., & Barlet-Ros, P. (2022). IGNNITION: Bridging the gap between graph neural networks and

networking systems. *IEEE Network*, *36*(1), 171–177. doi:10.1109/MNET.011.2100266

Roesch, M. (1999). Snort—Lightweight intrusion detection for networks. *Proceedings of the 13th USENIX Conference on System Administration* (pp. 229–238).

Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 108–116. doi:10.5220/0006639801080116

Simonovsky, M., & Komodakis, N. (2017). Dynamic edge-conditioned filters in convolutional neural networks on graphs. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 3693–3702). doi:10.1109/CVPR.2017.11

Tang, T. A., Mhamdi, L., McLernon, D., Zaidi, S. A. R., Ghogho, M., & El Moussa, F. (2022). DeepIDS: Deep learning approach for intrusion detection in software defined networking. *Electronics*, *9*(9), 1533. doi:10.3390/electronics9091533

Thakkar, A., & Lohiya, R. (2021). A review of the advancement in intrusion detection datasets. *Procedia Computer Science*, *167*, 636–645. doi:10.1016/j.procs.2020.03.330

Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. *Proceedings of the 6th International Conference on Learning Representations*.

Wang, Y., Li, J., Zhao, W., Han, Z., Zhao, H., Wang, L., & He, X. (2023). N-STGAT: Spatio-temporal graph neural network based network intrusion detection for near-earth remote sensing. *Remote Sensing*, *15*(14), 3611. doi:10.3390/rs15143611

Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., & Weinberger, K. (2019). Simplifying graph convolutional networks. *Proceedings of the 36th International Conference on Machine Learning*, *97*, 6861–6871.

Zhong, M., Lin, M., Zhang, C., & Xu, Z. (2024). A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. *Computers & Security*, *141*, 103821. doi:10.1016/j.cose.2024.103821

Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., ... Sun, M. (2020). Graph neural networks: A review of methods and applications. *AI Open*, *1*, 57–81. doi:10.1016/j.aiopen.2021.01.001